HIBERNATE ORM ZERO TO HERO INTERVIEW GUIDE

Java 17 • JPA 3.1 • Spring Data JPA

```java
@Entity
class User {
  @Id
  private Long id;
  private String name;
  private String email;
  @OneToMany
  private List<Order>
   orders;
}
```

```java
@Entity
class Order {
  @Id
  private Long id;
  private BigDecimal total;
  private LocalDateTime
  createdAt;
  @ManyToOne
  private User user;
}
```

ORM Mapping

Hibernate ORM
JPA 3.1

Generates SQL

Relational Database

1st Level Cache (Session)

2nd Level Cache (Optional)

Transaction Management (ACID)

users		
id	BIGINT	PK
name	VARCHAR(255)	
email	VARCHAR(255)	

orders		
id	BIGINT	PK
total	DECIMAL(10,2)	
created_at	TIMESTAMP	
user_id	BIGINT	FK

Entity Mapping and Relationships | JPQL and Criteria API | Transactions and Concurrency | Caching and Performance | Spring Data JPA Integration

100 QUESTIONS • 12 HANDS-ON LABS • 30-DAY PLAN

THE PREMIUM STUDY EDITION

How to use this guide

Start with the **direct answer**, trace the small code fragment, explain the diagram, then say the follow-up answer aloud. Study the invariant and the SQL it requires, not only annotation names.

What each interview page gives you

01. Precise interview answer in simple English

02. Mapping / lifecycle / query example

03. A color-coded concept flow

04. Practical trap and an answered follow-up

Teaching baseline

Java: 17+

ORM: Hibernate ORM 6.6

JPA: Jakarta Persistence 3.1

Spring: Spring Boot 3.x / Spring Data JPA

Database: SQL concepts; H2 for optional lab project

About these questions

This is an **original 100-question Hibernate interview bank**, created from scratch because no Hibernate source guide was supplied for this request. It does not claim that these exact questions were asked at a named company.

Code scope: Question snippets illustrate a specific idea and may omit imports/domain definitions. The coding ZIP contains a Maven project with 12 full Hibernate/JPA integration demonstrations, which must be run locally because Maven dependencies were unavailable in the creation workspace.

THE MAIN HIBERNATE MENTAL MODEL



COMPLETE CONTENTS

20 chapters / 100 questions

Everything from first entity mapping through senior architecture and tuning.

01 **ORM foundations and mental models**
Questions 001-005

02 **Entities, identifiers and value types**
Questions 006-010

03 **Entity lifecycle and state changes**
Questions 011-015

04 **Dirty checking, flush and write behavior**
Questions 016-020

05 **Relationships and ownership**
Questions 021-025

06 **Cascades and orphan removal**
Questions 026-030

07 **Fetching, proxies and LazyInitializationException**
Questions 031-035

08 **N+1 queries and fetch strategies**
Questions 036-040

09 **JPQL, HQL and DTO projections**
Questions 041-045

10 **Criteria API, filters and specifications**
Questions 046-050

11 **Transaction boundaries and Spring integration**
Questions 051-055

12 **Concurrency, isolation and locking**
Questions 056-060

13 **First-level cache and persistence context**
Questions 061-065

14 **Second-level and query caches**
Questions 066-070

15 **Batching, IDs and write optimization**
Questions 071-075

16 **Bulk updates, native SQL and consistency**
Questions 076-080

17 **Inheritance, polymorphism and mapping tradeoffs**
Questions 081-085

18 **Testing, schema migrations and verification**
Questions 086-090

19 **Observability and production performance**
Questions 091-095

20 **Senior architect scenarios and upgrades**
Questions 096-100

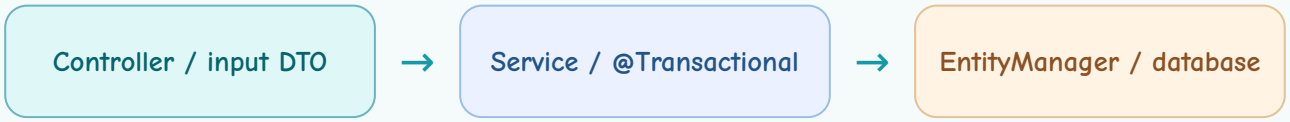
Also inside: 12 applied Hibernate labs, 30-day route, quick revision and primary references.
Question pages include diagrams, code, common pitfalls and individual follow-up answers.

ONE REAL REQUEST

Trace a request through Hibernate

The Java objects are not the database. Explain when an entity is tracked, when SQL runs, and what a transaction can truly guarantee.

REQUEST TO COMMIT



1. Authenticate and validate

The API establishes who can change which business object. Input constraints are checked before expensive work.

2. Load and modify managed data

The service loads the relevant entity, applies rules and relies on database constraints for concurrent correctness.

3. Flush is not the same as commit

Hibernate may send SQL before the transaction finishes. A rollback can still undo those changes.

4. Test the failure path

Try conflicting writers, retries, lost responses and deadlocks. Verify the SQL and affected rows.

Interview rule: State the invariant and its enforcement point. Example: “A seat can have only one confirmed owner because a database constraint rejects duplicate claims.”

01 / CHAPTER 01

ORM foundations and mental models

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q001 What problem does Hibernate ORM solve?

Q002 How do JPA and Hibernate differ?

Q003 What do EntityManager and Session represent?

Q004 How are EntityManagerFactory and EntityManager lifetimes different?

Q005 Why must you still design the database when using ORM?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / ORM FOUNDATIONS AND MENTAL MODELS

Q001
What problem does Hibernate ORM solve?

DIRECT ANSWER

Hibernate maps Java objects to relational rows and helps track changes, relationships and queries. It does not remove the need to understand SQL.

EXAMPLE / MAPPING OR QUERY

```
@Entity class Customer { @Id Long id; String name; }
```

TRACE THE IDEA



WATCH FOR

ORM is not a replacement for data modeling.

FOLLOW-UP QUESTION

Does ORM eliminate SQL tuning?

ANSWER

No. Hibernate still sends SQL. Inspect generated statements, indexes and execution plans when performance matters.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ORM FOUNDATIONS AND MENTAL MODELS

Q002
How do JPA and Hibernate differ?

DIRECT ANSWER

Jakarta Persistence defines standard mapping and persistence contracts. Hibernate ORM is one implementation that also offers provider-specific features.

EXAMPLE / MAPPING OR QUERY

```
EntityManager em; // JPA API, Hibernate provider
```

TRACE THE IDEA



WATCH FOR

A JPA annotation does not mean every provider generates identical SQL.

FOLLOW-UP QUESTION

When should you prefer standard JPA APIs?

ANSWER

Use EntityManager and standard annotations for portable behavior; use Hibernate extensions when they solve a concrete need.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ORM FOUNDATIONS AND MENTAL MODELS

Q003
What do EntityManager and Session represent?

DIRECT ANSWER

They manage a persistence context that tracks entity identities and changes. Hibernate Session provides extra native APIs alongside JPA EntityManager.

EXAMPLE / MAPPING OR QUERY

```
em.find(Customer.class, 42L);
```

TRACE THE IDEA



WATCH FOR

Do not store EntityManager in a singleton field.

FOLLOW-UP QUESTION

Can two threads share one EntityManager?

ANSWER

No. Treat an EntityManager or Session as a non-thread-safe unit of work and keep it scoped to one operation.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ORM FOUNDATIONS AND MENTAL MODELS

Q004

How are EntityManagerFactory and EntityManager lifetimes different?

DIRECT ANSWER

EntityManagerFactory is relatively costly and shared across application operations. EntityManager is lightweight, short-lived and not thread-safe.

EXAMPLE / MAPPING OR QUERY

```
try (var em = emf.createEntityManager()) { /* work */ }
```

TRACE THE IDEA



WATCH FOR

Opening a new factory for every request wastes resources.

FOLLOW-UP QUESTION

Who closes these resources?

ANSWER

The container normally manages them in Spring; in plain Java, close each EntityManager and close the factory at application shutdown.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ORM FOUNDATIONS AND MENTAL MODELS

Q005

Why must you still design the database when using ORM?

DIRECT ANSWER

Tables, constraints, types and indexes enforce important rules and determine query cost. Entity annotations are only one representation of that schema.

EXAMPLE / MAPPING OR QUERY

```
@Column(nullable=false) String email; // also use DB constraints
```

TRACE THE IDEA



WATCH FOR

Do not rely on Java validation alone under concurrent writes.

FOLLOW-UP QUESTION

Which rule belongs in the database?

ANSWER

Business uniqueness, required foreign keys and nonnegative stock constraints should be enforced by suitable database mechanisms.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

02 / CHAPTER 02

Entities, identifiers and value types

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q006 Which Java class can be a JPA entity?

Q007 How do @Id and @GeneratedValue work?

Q008 When should you use @Embedded and @Embeddable?

Q009 How should enum values be stored?

Q010 When should you use an AttributeConverter?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / ENTITIES, IDENTIFIERS AND VALUE TYPES

Q006
Which Java class can be a JPA entity?

DIRECT ANSWER

An entity is mapped with @Entity and has an identifier. For portable JPA, provide an accessible no-argument constructor and a non-final entity class.

EXAMPLE / MAPPING OR QUERY

```
@Entity class Order { @Id Long id; protected Order() {} }
```

TRACE THE IDEA

identifier → entity mapping → persisted row

WATCH FOR

Do not assume every immutable record can be a JPA entity.

FOLLOW-UP QUESTION

Why is the no-arg constructor needed?

ANSWER

JPA providers instantiate entities reflectively or through enhancement; keep a protected constructor for domain control.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITIES, IDENTIFIERS AND VALUE TYPES

Q007
How do @Id and @GeneratedValue work?

DIRECT ANSWER

@Id marks the persistent identity. @GeneratedValue asks the provider or database to generate identifiers under a selected strategy.

EXAMPLE / MAPPING OR QUERY

```
@Id @GeneratedValue(strategy=GenerationType.IDENTITY) Long id;
```

TRACE THE IDEA



WATCH FOR

Do not assume an identifier exists before the insert happens.

FOLLOW-UP QUESTION

What is the tradeoff of IDENTITY generation?

ANSWER

IDENTITY usually needs a database insert to obtain the identifier, which can limit JDBC insert batching.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITIES, IDENTIFIERS AND VALUE TYPES

Q008

When should you use @Embedded and @Embeddable?

DIRECT ANSWER

Use an embeddable for value fields that belong to the owning row or mapping. It has no separate entity identity.

EXAMPLE / MAPPING OR QUERY

```
@Embedded Address shippingAddress;
```

TRACE THE IDEA



WATCH FOR

Value objects can still hold mutable references unless carefully designed.

FOLLOW-UP QUESTION

Should an Address be an entity or value?

ANSWER

Make it an embeddable when it has no independent lifecycle; make it an entity when it must be shared and independently updated.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITIES, IDENTIFIERS AND VALUE TYPES

Q009
How should enum values be stored?

DIRECT ANSWER

Prefer EnumType.STRING for readable stable names, and plan migrations before renaming constants. ORDINAL depends on declaration order.

EXAMPLE / MAPPING OR QUERY

```
@Enumerated(EnumType.STRING) OrderStatus status;
```

TRACE THE IDEA



WATCH FOR

Reordering ORDINAL values can silently change meaning.

FOLLOW-UP QUESTION

What if an enum constant must be renamed?

ANSWER

Use an explicit converter or migration so persisted codes remain compatible.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITIES, IDENTIFIERS AND VALUE TYPES

Q010
When should you use an AttributeConverter?

DIRECT ANSWER

Use it to translate a domain value to an agreed database representation when basic mapping is insufficient.

EXAMPLE / MAPPING OR QUERY

```
@Converter class CurrencyCodeConverter implements
AttributeConverter<CurrencyCode,String> { ... }
```

TRACE THE IDEA



WATCH FOR

Converters do not automatically make ciphertext searchable or secure.

FOLLOW-UP QUESTION

Can one converter handle encrypted secrets transparently?

ANSWER

Technically conversion is possible, but encryption needs keys, rotation and query constraints designed separately.

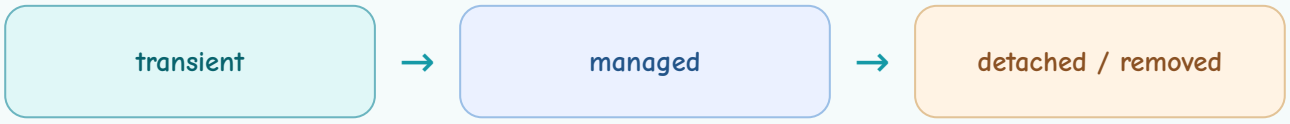
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

03 / CHAPTER 03

Entity lifecycle and state changes

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q011 What are transient, managed, detached and removed entity states?

Q012 What does persist() do?

Q013 How does merge() differ from persist()?

Q014 What does remove() mean for a managed entity?

Q015 What are detach(), clear() and refresh() used for?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / ENTITY LIFECYCLE AND STATE CHANGES

Q011

What are transient, managed, detached and removed entity states?

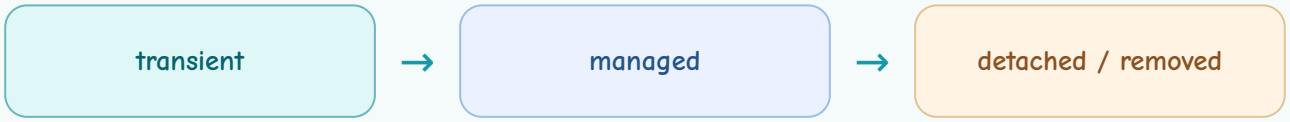
DIRECT ANSWER

A new object is transient; a managed entity is tracked; a detached entity is no longer tracked; a removed entity is scheduled for deletion.

EXAMPLE / MAPPING OR QUERY

```
Customer c = new Customer(); em.persist(c); // managed
```

TRACE THE IDEA



WATCH FOR

Avoid equating a Java object reference with active persistence tracking.

FOLLOW-UP QUESTION

Can a detached object's setter update the database?

ANSWER

No. Changes to a detached object are not tracked unless they are merged or explicitly reattached using an appropriate operation.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITY LIFECYCLE AND STATE CHANGES

Q012
What does persist() do?

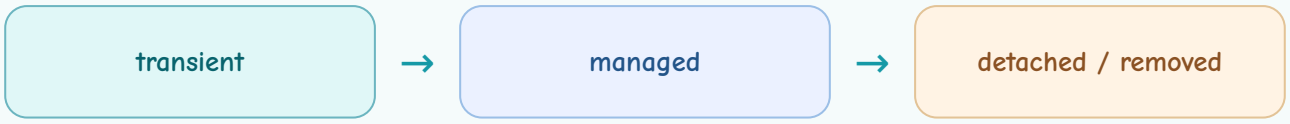
DIRECT ANSWER

persist makes a new entity managed and schedules insertion according to flush and identifier rules.

EXAMPLE / MAPPING OR QUERY

```
em.persist(new Customer("Meena"));
```

TRACE THE IDEA



WATCH FOR

Persist is not an independent durable commit.

FOLLOW-UP QUESTION

Does persist always issue INSERT immediately?

ANSWER

No. SQL timing depends on the identifier generator and flush requirements; commit normally triggers flushing.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITY LIFECYCLE AND STATE CHANGES

Q013
How does merge() differ from persist()?

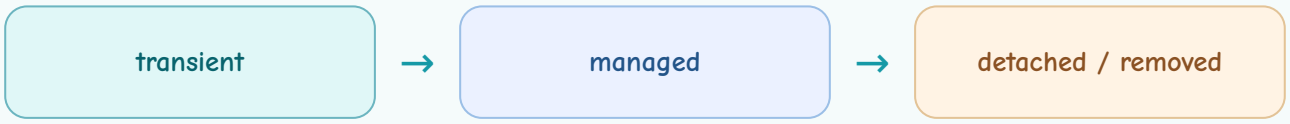
DIRECT ANSWER

merge copies the state of a detached or new object into a managed instance and returns that managed instance. The input reference does not become managed.

EXAMPLE / MAPPING OR QUERY

```
Customer managed = em.merge(detached);
```

TRACE THE IDEA



WATCH FOR

Ignoring the return value can cause stale-reference mistakes.

FOLLOW-UP QUESTION

Which object should you use after merge?

ANSWER

Use the returned object; it is the managed result in the current persistence context.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITY LIFECYCLE AND STATE CHANGES

Q014

What does remove() mean for a managed entity?

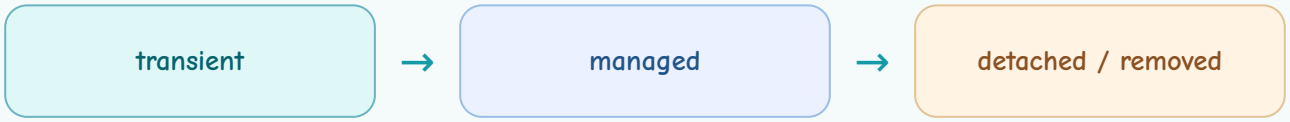
DIRECT ANSWER

remove marks a managed entity for deletion at flush, subject to mappings and database constraints.

EXAMPLE / MAPPING OR QUERY

```
em.remove(em.find(Customer.class, id));
```

TRACE THE IDEA



WATCH FOR

Cascade REMOVE can erase more rows than intended.

FOLLOW-UP QUESTION

Can you remove a detached entity directly?

ANSWER

Not portably. Find or merge an appropriate managed instance first, and check authorization before deleting.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / ENTITY LIFECYCLE AND STATE CHANGES

Q015

What are detach(), clear() and refresh() used for?

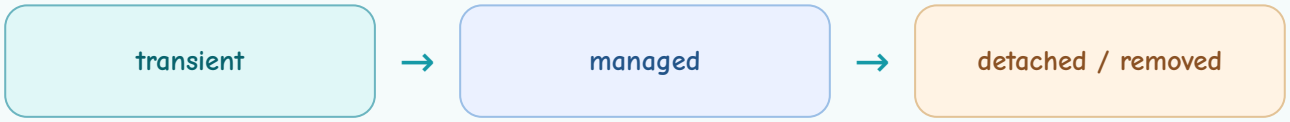
DIRECT ANSWER

detach stops tracking one entity; clear detaches all managed entities; refresh reloads database state into a managed entity.

EXAMPLE / MAPPING OR QUERY

```
em.flush(); em.clear();
```

TRACE THE IDEA



WATCH FOR

Clearing is not the same as committing a transaction.

FOLLOW-UP QUESTION

Why can clear() lose work?

ANSWER

Unflushed changes are discarded from tracking, so flush intentionally before clearing when those updates must persist.

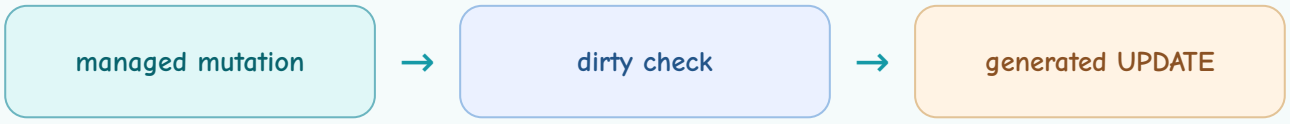
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

04 / CHAPTER 04

Dirty checking, flush and write behavior

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q016 How does Hibernate dirty checking work?

Q017 What is the difference between flush and commit?

Q018 When does Hibernate flush automatically?

Q019 What does @DynamicUpdate change?

Q020 Why does write ordering matter during flush?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / DIRTY CHECKING, FLUSH AND WRITE BEHAVIOR

Q016
How does Hibernate dirty checking work?

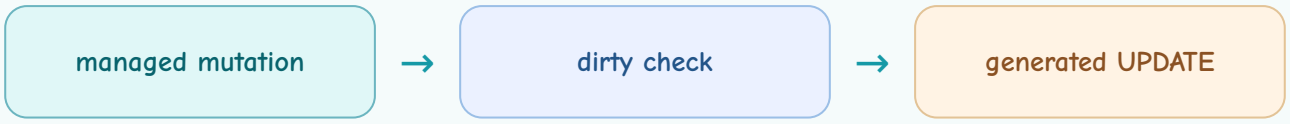
DIRECT ANSWER

Hibernate compares or tracks a managed entity's state and generates writes when flushing changes.

EXAMPLE / MAPPING OR QUERY

```
Customer c=em.find(Customer.class,id); c.setName("New");
```

TRACE THE IDEA



WATCH FOR

Changing a detached entity will not be picked up automatically.

FOLLOW-UP QUESTION

Do you always need save() after editing a managed entity?

ANSWER

No. Within a writable transaction, dirty checking can persist the change during flush without another repository save call.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / DIRTY CHECKING, FLUSH AND WRITE BEHAVIOR

Q017
What is the difference between flush and commit?

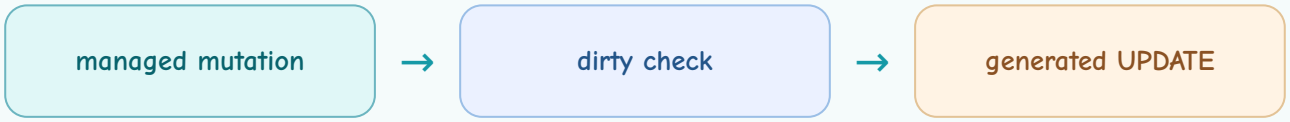
DIRECT ANSWER

Flush synchronizes pending persistence operations to the database inside the current transaction. Commit decides whether the transaction becomes durable.

EXAMPLE / MAPPING OR QUERY

```
em.persist(order); em.flush(); tx.rollback();
```

TRACE THE IDEA



WATCH FOR

A successful flush is not proof of business success.

FOLLOW-UP QUESTION

Can a flushed INSERT still roll back?

ANSWER

Yes. A later transaction rollback can undo flushed database writes.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / DIRTY CHECKING, FLUSH AND WRITE BEHAVIOR

Q018
When does Hibernate flush automatically?

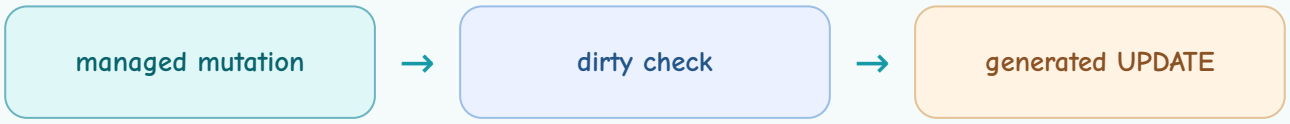
DIRECT ANSWER

Flush can happen before transaction commit and before some queries whose results require pending changes, depending on flush mode and provider behavior.

EXAMPLE / MAPPING OR QUERY

```
em.setFlushMode(FlushModeType.AUTO);
```

TRACE THE IDEA



WATCH FOR

Do not rely on accidental query-triggered flushing.

FOLLOW-UP QUESTION

Can flush mode affect queries?

ANSWER

Yes. AUTO and COMMIT modes can produce different synchronization timing; validate actual query behavior.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / DIRTY CHECKING, FLUSH AND WRITE BEHAVIOR

Q019
What does @DynamicUpdate change?

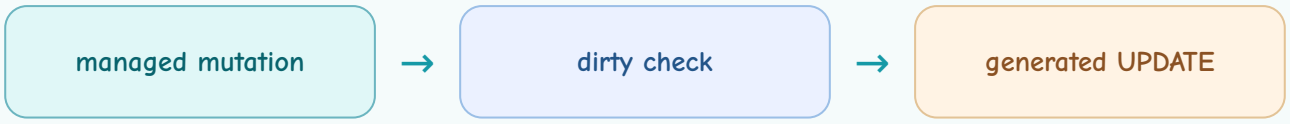
DIRECT ANSWER

It asks Hibernate to generate UPDATE SQL with changed columns when possible, trading SQL-statement variation for narrower updates.

EXAMPLE / MAPPING OR QUERY

```
@org.hibernate.annotations.DynamicUpdate
```

TRACE THE IDEA



WATCH FOR

Narrow UPDATEs can reduce batching and statement-plan reuse.

FOLLOW-UP QUESTION

Does it eliminate optimistic lock needs?

ANSWER

No. It does not provide cross-request conflict detection or protect an invariant by itself.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / DIRTY CHECKING, FLUSH AND WRITE BEHAVIOR

Q020
Why does write ordering matter during flush?

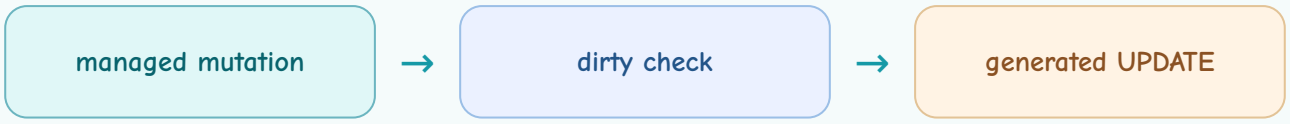
DIRECT ANSWER

Insert, update and delete actions may execute in an order chosen to satisfy batching and persistence rules, not the order of Java setter calls.

EXAMPLE / MAPPING OR QUERY

```
em.flush(); // verify SQL and constraint timing
```

TRACE THE IDEA



WATCH FOR

Do not assume Java statement order equals SQL execution order.

FOLLOW-UP QUESTION

How can a unique-key swap fail?

ANSWER

Two updates may conflict mid-flush; design temporary values or database-supported deferrable constraints where available.

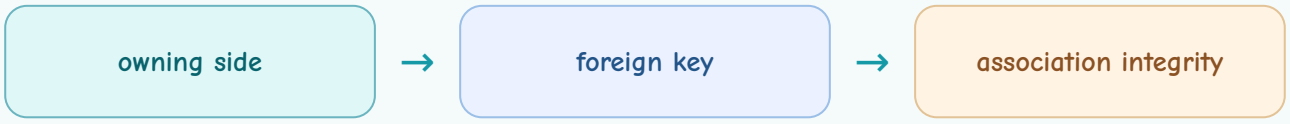
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

05 / CHAPTER 05

Relationships and ownership

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q021 How do @ManyToOne and @OneToMany differ?

Q022 What does mappedBy mean?

Q023 When should you use @OneToOne?

Q024 Why is @ManyToMany often avoided for rich domains?

Q025 What is a useful helper method for bidirectional associations?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / RELATIONSHIPS AND OWNERSHIP

Q021
How do @ManyToOne and @OneToMany differ?

DIRECT ANSWER

Many orders can refer to one customer via a foreign key. A collection on the customer side represents the inverse one-to-many view.

EXAMPLE / MAPPING OR QUERY

```
@ManyToOne(fetch=FetchType.LAZY) Customer customer;
```

TRACE THE IDEA

owning side → foreign key → association integrity

WATCH FOR

Do not assume the collection is the database owner.

FOLLOW-UP QUESTION

Which side normally owns the foreign key?

ANSWER

The @ManyToOne side typically holds and updates the foreign key column.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / RELATIONSHIPS AND OWNERSHIP

Q022
What does mappedBy mean?

DIRECT ANSWER

mappedBy identifies the field that owns a bidirectional relationship; the inverse side does not maintain the foreign key independently.

EXAMPLE / MAPPING OR QUERY

```
customer.addOrder(order); // sets order.customer and collection
```

TRACE THE IDEA

owning side → foreign key → association integrity

WATCH FOR

Updating only inverse collection can fail to change the FK.

FOLLOW-UP QUESTION

Do both Java references need updating?

ANSWER

Yes for a consistent in-memory object graph, even though only the owning side controls persistence.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / RELATIONSHIPS AND OWNERSHIP

Q023
When should you use @OneToOne?

DIRECT ANSWER

Use it when each row relates to at most one other row under an enforced uniqueness rule.

EXAMPLE / MAPPING OR QUERY

```
@OneToOne @JoinColumn(name="passport_id", unique=true)
```

TRACE THE IDEA

owning side → foreign key → association integrity

WATCH FOR

A missing unique constraint can violate the assumed one-to-one rule.

FOLLOW-UP QUESTION

Does @OneToOne automatically guarantee a unique business relation?

ANSWER

Only with the correct schema constraints and mapping; verify the generated database definition.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / RELATIONSHIPS AND OWNERSHIP

Q024

Why is @ManyToMany often avoided for rich domains?

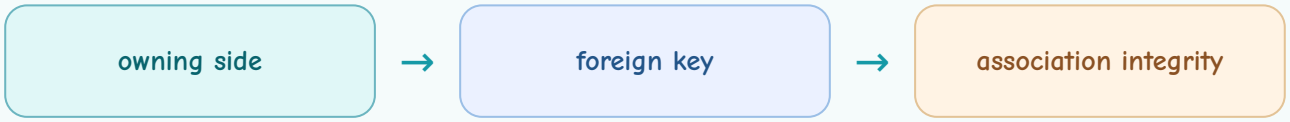
DIRECT ANSWER

The join table usually becomes a real domain relationship when it needs timestamps, status or ownership rules. Map that join row as an entity.

EXAMPLE / MAPPING OR QUERY

```
class Enrollment { Student student; Course course; LocalDate enrolledAt; }
```

TRACE THE IDEA



WATCH FOR

A bare @ManyToMany cannot carry join-row fields naturally.

FOLLOW-UP QUESTION

How would you store enrollment date?

ANSWER

Create an Enrollment entity with @ManyToOne references to Student and Course.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / RELATIONSHIPS AND OWNERSHIP

Q025

What is a useful helper method for bidirectional associations?

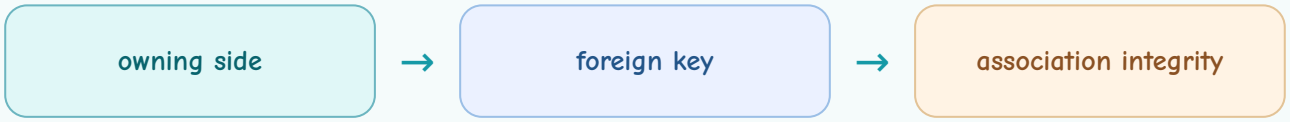
DIRECT ANSWER

A helper changes the owning field and the inverse collection together so the in-memory graph stays coherent.

EXAMPLE / MAPPING OR QUERY

```
void addOrder(Order o){orders.add(o);o.setCustomer(this);}
```

TRACE THE IDEA



WATCH FOR

Watch for recursive equals/toString across both sides.

FOLLOW-UP QUESTION

Is a helper sufficient to enforce database integrity?

ANSWER

No. Foreign keys and uniqueness constraints remain necessary.

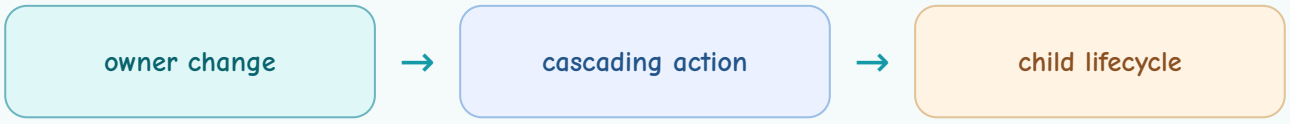
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

06 / CHAPTER 06

Cascades and orphan removal

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q026 What does CascadeType.PERSIST do?

Q027 How do cascade and orphanRemoval differ?

Q028 Why is CascadeType.REMOVE risky on @ManyToMany?

Q029 Why might a cascade seem to do nothing?

Q030 How should aggregate boundaries guide cascades?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / CASCADES AND ORPHAN REMOVAL

Q026
What does CascadeType.PERSIST do?

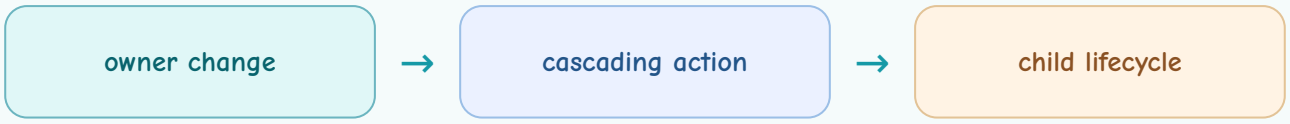
DIRECT ANSWER

It propagates persist from a parent to related objects through a mapped association. Only configure it when the parent controls that creation lifecycle.

EXAMPLE / MAPPING OR QUERY

```
@OneToMany(cascade= CascadeType.PERSIST) List<Item> items;
```

TRACE THE IDEA



WATCH FOR

ALL is not a default that fits every association.

FOLLOW-UP QUESTION

Should you cascade from child to shared parent?

ANSWER

Usually no. Persisting one order should not automatically create unrelated customer records.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CASCADES AND ORPHAN REMOVAL

Q027
How do cascade and orphanRemoval differ?

DIRECT ANSWER

Cascade REMOVE propagates deletion when the parent is removed. orphanRemoval deletes a child when it is removed from its owning collection/ relationship.

EXAMPLE / MAPPING OR QUERY

```
@OneToMany(mappedBy="order", orphanRemoval=true) List<Item> items;
```

TRACE THE IDEA



WATCH FOR

Removing a list entry can delete a database row.

FOLLOW-UP QUESTION

What if a child is merely moved to another parent?

ANSWER

Handle the move explicitly and verify ownership rules; orphan removal can schedule deletion if modeled incorrectly.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CASCADES AND ORPHAN REMOVAL

Q028

Why is CascadeType.REMOVE risky on @ManyToMany?

DIRECT ANSWER

Related entities are usually shared by many owners. Cascading delete could remove another owner's shared entity.

EXAMPLE / MAPPING OR QUERY

```
@ManyToMany Set<Tag> tags; // no REMOVE cascade
```

TRACE THE IDEA



WATCH FOR

Shared reference data must not be cascaded away.

FOLLOW-UP QUESTION

What should deletion remove in a many-to-many relation?

ANSWER

Remove the join association, not the shared peer entity.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CASCADES AND ORPHAN REMOVAL

Q029
Why might a cascade seem to do nothing?

DIRECT ANSWER

Cascade directions only follow configured paths; the parent must be in the relevant operation/state and associations must be synchronized.

EXAMPLE / MAPPING OR QUERY

```
em.persist(order); // inspect new item insert
```

TRACE THE IDEA

owner change → cascading action → child lifecycle

WATCH FOR

Cascade is not equivalent to unlimited graph persistence.

FOLLOW-UP QUESTION

How do you debug it?

ANSWER

Inspect owning side, cascade configuration, transaction, flush and SQL output.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CASCADES AND ORPHAN REMOVAL

Q030

How should aggregate boundaries guide cascades?

DIRECT ANSWER

Use cascading for children whose lifecycle genuinely belongs to one aggregate root. Shared external aggregates should be addressed explicitly.

EXAMPLE / MAPPING OR QUERY

```
Order -> OrderItem : owned; Order -> Customer : reference
```

TRACE THE IDEA



WATCH FOR

Mapping convenience must not define business ownership.

FOLLOW-UP QUESTION

Should checkout cascade updates into Customer?

ANSWER

Not merely because an Order references Customer; ownership is a business design choice.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

07 / CHAPTER 07

Fetching, proxies and LazyInitializationException

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q031 What does LAZY fetching mean?

Q032 What causes LazyInitializationException?

Q033 What is an entity proxy?

Q034 When should you use an EntityGraph?

Q035 What should a REST controller return instead of entities?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / FETCHING, PROXIES AND LAZYINITIALIZATIONEXCEPTION

Q031
What does LAZY fetching mean?

DIRECT ANSWER

Hibernate may defer association loading until it is accessed in an open persistence context. It is a fetch hint/contract subject to provider and mapping rules.

EXAMPLE / MAPPING OR QUERY

```
@ManyToOne(fetch=FetchType.LAZY) Customer customer;
```

TRACE THE IDEA



WATCH FOR

Never assume lazy means no database call during serialization.

FOLLOW-UP QUESTION

Does LAZY guarantee zero SQL at initial load?

ANSWER

No. Mapping and instrumentation details can affect whether an association is fetched eagerly.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FETCHING, PROXIES AND LAZYINITIALIZATIONEXCEPTION

Q032
What causes LazyInitializationException?

DIRECT ANSWER

Code accesses an uninitialized lazy association after its persistence context is unavailable.

EXAMPLE / MAPPING OR QUERY

```
List<OrderView> getOrders(); // map within transaction
```

TRACE THE IDEA



WATCH FOR

Do not fix it by making every relationship EAGER.

FOLLOW-UP QUESTION

Is Open Session in View the ideal fix?

ANSWER

Not necessarily. Prefer transaction-scoped queries and intentional DTO fetch plans; OSIV can hide extra SQL and extend persistence work.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FETCHING, PROXIES AND LAZYINITIALIZATIONEXCEPTION

Q033
What is an entity proxy?

DIRECT ANSWER

A proxy can stand in for an entity and load its state when needed. Its concrete runtime type may differ from the mapped class.

EXAMPLE / MAPPING OR QUERY

```
Customer ref = em.getReference(Customer.class,id);
```

TRACE THE IDEA



WATCH FOR

Avoid dereferencing a proxy outside its context.

FOLLOW-UP QUESTION

Why can getClass() and equals() be tricky?

ANSWER

Proxy subclasses and uninitialized identifiers complicate entity equality; use a carefully tested identity strategy.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FETCHING, PROXIES AND LAZYINITIALIZATIONEXCEPTION

Q034
When should you use an EntityGraph?

DIRECT ANSWER

An EntityGraph declares which relationships should be included for a particular operation without changing all mappings globally.

EXAMPLE / MAPPING OR QUERY

```
em.find(Order.class,id, Map.of("jakarta.persistence.fetchgraph",graph));
```

TRACE THE IDEA



WATCH FOR

Fetch graphs must be designed for query shape and cardinality.

FOLLOW-UP QUESTION

Is an EntityGraph identical to SQL JOIN FETCH?

ANSWER

No. A provider can choose how to implement a fetch plan; inspect resulting SQL and duplicates.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FETCHING, PROXIES AND LAZYINITIALIZATIONEXCEPTION

Q035

What should a REST controller return instead of entities?

DIRECT ANSWER

Return DTOs representing the API contract, with data loaded intentionally inside a service transaction.

EXAMPLE / MAPPING OR QUERY

```
record OrderDto(Long id, String status) {}
```

TRACE THE IDEA



WATCH FOR

A JSON annotation does not substitute for a proper fetch plan.

FOLLOW-UP QUESTION

What risk comes from serializing entities directly?

ANSWER

Lazy loading, recursion, leaking fields and accidental deep graph queries.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

08 / CHAPTER 08

N+1 queries and fetch strategies

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q036 What is the N+1 select problem?

Q037 How can JOIN FETCH help?

Q038 What does @BatchSize optimize?

Q039 What is subselect fetching?

Q040 How should you fetch data for a paginated list?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / N+1 QUERIES AND FETCH STRATEGIES

Q036
What is the N+1 select problem?

DIRECT ANSWER

A query loads N parents and then causes another query for each parent's relationship, producing 1+N round trips.

EXAMPLE / MAPPING OR QUERY

```
orders.forEach(o -> o.getCustomer().getName());
```

TRACE THE IDEA

initial SELECT → repeated child load → bounded query plan

WATCH FOR

A method that works with 3 rows can become slow with 3,000.

FOLLOW-UP QUESTION

How can you detect N+1?

ANSWER

Turn on SQL/statistics in tests and assert statement counts for representative fixture sizes.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / N+1 QUERIES AND FETCH STRATEGIES

Q037
How can JOIN FETCH help?

DIRECT ANSWER

It requests association data in the same query where appropriate, reducing round trips for the selected graph.

EXAMPLE / MAPPING OR QUERY

```
select o from Order o join fetch o.customer
```

TRACE THE IDEA

initial SELECT → repeated child load → bounded query plan

WATCH FOR

A join fetch can inflate result rows and transferred bytes.

FOLLOW-UP QUESTION

Why is joining multiple collections dangerous?

ANSWER

Joining two bags can be unsupported or multiply rows; bound collection cardinality and consider multiple queries.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / N+1 QUERIES AND FETCH STRATEGIES

Q038
What does @BatchSize optimize?

DIRECT ANSWER

Batch fetching gathers several uninitialized associations into fewer SELECTs, often with an IN-list.

EXAMPLE / MAPPING OR QUERY

```
@BatchSize(size=32) List<OrderItem> items;
```

TRACE THE IDEA

initial SELECT → repeated child load → bounded query plan

WATCH FOR

Batching reduces trips but may still load unused data.

FOLLOW-UP QUESTION

Is batch size 100 always better than 20?

ANSWER

No. Tune against DB parameter limits, network cost and typical association counts.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / N+1 QUERIES AND FETCH STRATEGIES

Q039
What is subselect fetching?

DIRECT ANSWER

Hibernate can fetch collections for multiple loaded parents using a subquery derived from the parent-loading scope.

EXAMPLE / MAPPING OR QUERY

```
@Fetch(FetchMode.SUBSELECT) List<Item> items;
```

TRACE THE IDEA

initial SELECT → repeated child load → bounded query plan

WATCH FOR

Subselect fetch is not a universal N+1 fix.

FOLLOW-UP QUESTION

When is it a bad choice?

ANSWER

When a large parent result forces fetching many collections not needed by the screen.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / N+1 QUERIES AND FETCH STRATEGIES

Q040

How should you fetch data for a paginated list?

DIRECT ANSWER

Page root IDs or DTO rows first, then fetch bounded needed details using a second query if necessary.

EXAMPLE / MAPPING OR QUERY

```
select o.id from Order o order by o.id
```

TRACE THE IDEA

initial SELECT → repeated child load → bounded query plan

WATCH FOR

A warning about in-memory pagination should not be ignored.

FOLLOW-UP QUESTION

Why avoid paging directly over a collection fetch join?

ANSWER

SQL row multiplication can make root pagination incorrect or force expensive in-memory paging.

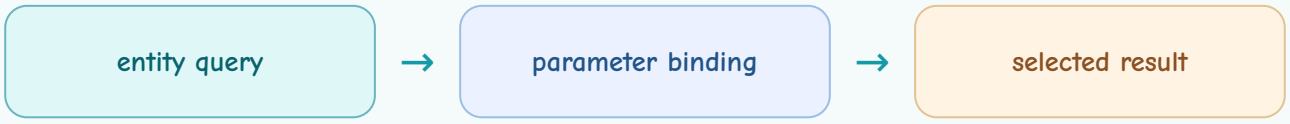
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

09 / CHAPTER 09

JPQL, HQL and DTO projections

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q041 What is JPQL and how is it different from SQL?

Q042 How do you bind query parameters safely?

Q043 What are DTO constructor projections?

Q044 What is the difference between getResultList and getSingleResult?

Q045 When do you use a native SQL query?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / JPQL, HQL AND DTO PROJECTIONS

Q041
What is JPQL and how is it different from SQL?

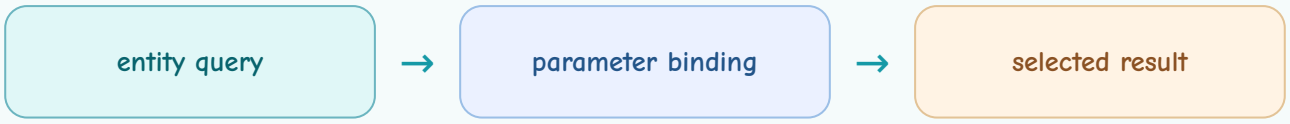
DIRECT ANSWER

JPQL queries the entity model and its fields, while SQL queries database tables and columns. Hibernate HQL adds provider features.

EXAMPLE / MAPPING OR QUERY

```
select o from Order o where o.customer.id = :id
```

TRACE THE IDEA



WATCH FOR

JPQL and SQL are not interchangeable text.

FOLLOW-UP QUESTION

Will JPQL naming match physical column names?

ANSWER

No. Use Java entity property names and associations.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / JPQL, HQL AND DTO PROJECTIONS

Q042
How do you bind query parameters safely?

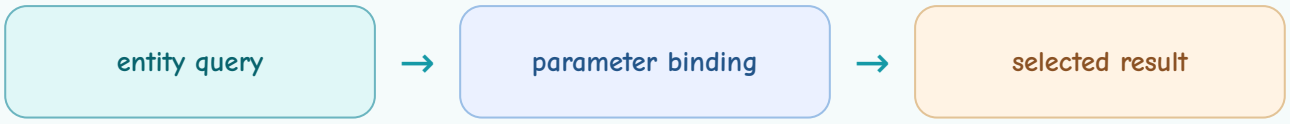
DIRECT ANSWER

Use typed or named parameters rather than concatenating untrusted values into JPQL/SQL.

EXAMPLE / MAPPING OR QUERY

```
query.setParameter("customerId", customerId);
```

TRACE THE IDEA



WATCH FOR

Never concatenate user-supplied filtering values.

FOLLOW-UP QUESTION

Does a bound parameter substitute for a table name?

ANSWER

No. Whitelist any dynamic identifiers separately; parameters represent values.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / JPQL, HQL AND DTO PROJECTIONS

Q043
What are DTO constructor projections?

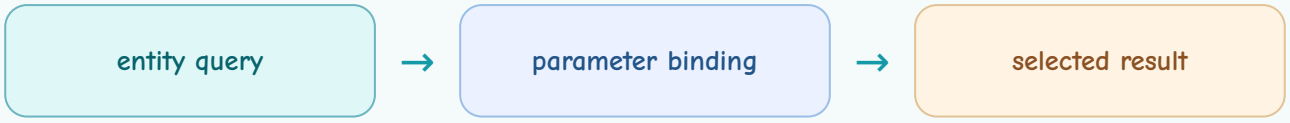
DIRECT ANSWER

A constructor projection selects only needed fields into a class with a matching constructor instead of a managed entity.

EXAMPLE / MAPPING OR QUERY

```
select new app.OrderSummary(o.id,o.total) from Order o
```

TRACE THE IDEA



WATCH FOR

A DTO projection is not managed and won't be dirty-checked.

FOLLOW-UP QUESTION

Why use a DTO for a dashboard?

ANSWER

It reduces entity materialization and prevents accidental lazy loading for read-only views.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / JPQL, HQL AND DTO PROJECTIONS

Q044

What is the difference between getResultList and getSingleResult?

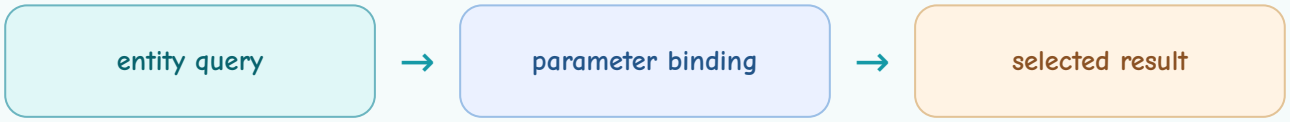
DIRECT ANSWER

getResultList returns zero or more results; getSingleResult expects one and throws for no result or more than one.

EXAMPLE / MAPPING OR QUERY

```
query.setMaxResults(1).getResultList();
```

TRACE THE IDEA



WATCH FOR

LIMIT 1 without ORDER BY can choose an arbitrary match.

FOLLOW-UP QUESTION

How would you handle optional lookup?

ANSWER

Use an appropriate bounded list or supported convenience API and encode absence explicitly.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / JPQL, HQL AND DTO PROJECTIONS

Q045
When do you use a native SQL query?

DIRECT ANSWER

Use it when a database-specific feature or proven query requirement cannot be expressed well through JPQL/HQL.

EXAMPLE / MAPPING OR QUERY

```
em.createNativeQuery("select ...");
```

TRACE THE IDEA

entity query → parameter binding → selected result

WATCH FOR

Native SQL still needs binding, authorization and review.

FOLLOW-UP QUESTION

What is the portability cost?

ANSWER

Schema names, result mapping, dialect functions and behavior can change by database.

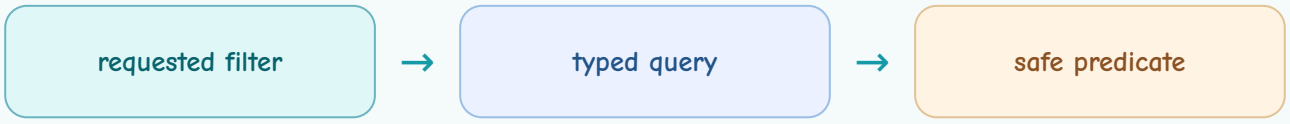
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

10 / CHAPTER 10

Criteria API, filters and specifications

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q046 What is the JPA Criteria API?

Q047 How can optional filters be composed?

Q048 What is Spring Data JPA Specification?

Q049 How do you implement safe dynamic sorting?

Q050 How do you apply multi-tenant predicates?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / CRITERIA API, FILTERS AND SPECIFICATIONS

Q046
What is the JPA Criteria API?

DIRECT ANSWER

Criteria lets Java code construct an entity query dynamically using typed expression objects.

EXAMPLE / MAPPING OR QUERY

```
CriteriaBuilder cb=em.getCriteriaBuilder();
```

TRACE THE IDEA

requested filter → typed query → safe predicate

WATCH FOR

Criteria is not automatically faster than JPQL.

FOLLOW-UP QUESTION

When is it better than a constant JPQL string?

ANSWER

When optional filters and composition justify the extra verbosity.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CRITERIA API, FILTERS AND SPECIFICATIONS

Q047
How can optional filters be composed?

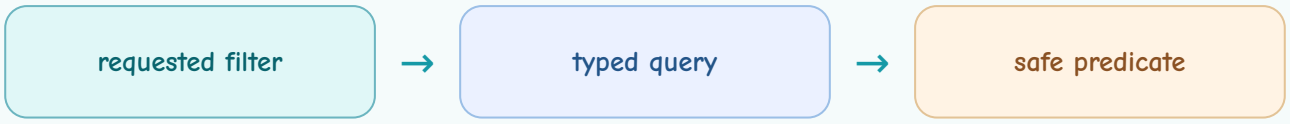
DIRECT ANSWER

Add only active predicates and combine them with CriteriaBuilder.and/or according to the requested semantics.

EXAMPLE / MAPPING OR QUERY

```
List<Predicate> parts=new ArrayList<>();
```

TRACE THE IDEA



WATCH FOR

An absent predicate must not bypass tenant isolation.

FOLLOW-UP QUESTION

What happens with no filters?

ANSWER

Choose whether to return a bounded default result or reject an unbounded query.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CRITERIA API, FILTERS AND SPECIFICATIONS

Q048
What is Spring Data JPA Specification?

DIRECT ANSWER

A Specification composes JPA Criteria predicates for Spring Data repository queries.

EXAMPLE / MAPPING OR QUERY

```
Specification<Order> byStatus = (root,q,cb) -> cb.equal(root.get("status"),status);
```

TRACE THE IDEA

requested filter → typed query → safe predicate

WATCH FOR

Dynamic criteria can still produce bad indexes or N+1 queries.

FOLLOW-UP QUESTION

Does Specification change persistence-provider rules?

ANSWER

No. It builds queries that Hibernate still executes within ORM constraints.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CRITERIA API, FILTERS AND SPECIFICATIONS

Q049
How do you implement safe dynamic sorting?

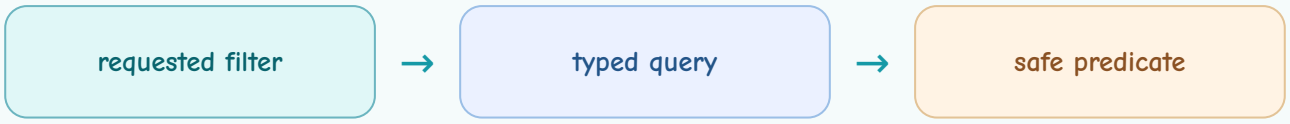
DIRECT ANSWER

Map an allowlisted API sort field to a known entity path, then use a stable tie-breaker.

EXAMPLE / MAPPING OR QUERY

```
ORDER BY createdAt DESC, id DESC
```

TRACE THE IDEA



WATCH FOR

Stable ordering is mandatory for reliable pagination.

FOLLOW-UP QUESTION

Why should sort expressions be allowlisted?

ANSWER

Unrestricted paths can create expensive joins or expose unauthorized fields.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CRITERIA API, FILTERS AND SPECIFICATIONS

Q050
How do you apply multi-tenant predicates?

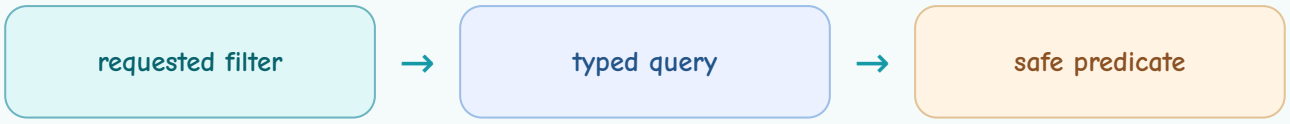
DIRECT ANSWER

Apply tenant scope on every relevant query path and validate ownership at the service boundary.

EXAMPLE / MAPPING OR QUERY

```
where order.tenantId = :authenticatedTenant
```

TRACE THE IDEA



WATCH FOR

Do not trust a tenant ID supplied only by the client.

FOLLOW-UP QUESTION

Are repository filters enough for hard isolation?

ANSWER

Not always. Bulk/native queries and bypass paths need review; defense-in-depth via DB policies may be appropriate.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

11 / CHAPTER 11

Transaction boundaries and Spring integration

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q051 What does @Transactional do in a Spring JPA service?

Q052 Why should a transaction be short?

Q053 How do checked and unchecked exceptions affect Spring rollback?

Q054 What does readOnly=true mean?

Q055 Why can transaction self-invocation fail?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / TRANSACTION BOUNDARIES AND SPRING INTEGRATION

Q051

What does @Transactional do in a Spring JPA service?

DIRECT ANSWER

When properly intercepted, it opens or joins a transaction and manages commit or rollback according to the transaction manager and rules.

EXAMPLE / MAPPING OR QUERY

```
@Transactional public void placeOrder(...) { ... }
```

TRACE THE IDEA



WATCH FOR

A method annotation alone is not proof of a transaction.

FOLLOW-UP QUESTION

Does @Transactional on a private method work by default?

ANSWER

No. Default proxy advice usually applies to eligible calls through a Spring proxy, not internal self-invocation.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TRANSACTION BOUNDARIES AND SPRING INTEGRATION

Q052
Why should a transaction be short?

DIRECT ANSWER

Long transactions hold locks and database connections, increase contention and can keep stale snapshots alive.

EXAMPLE / MAPPING OR QUERY

```
reserveLocally(); // commit; then coordinate payment
```

TRACE THE IDEA



WATCH FOR

A database transaction cannot atomically cover an unrelated remote API.

FOLLOW-UP QUESTION

Should you make an HTTP payment call inside a DB transaction?

ANSWER

Usually avoid it. Use a durable workflow with explicit state and recovery across independent systems.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TRANSACTION BOUNDARIES AND SPRING INTEGRATION

Q053

How do checked and unchecked exceptions affect Spring rollback?

DIRECT ANSWER

By default Spring rolls back on RuntimeException and Error, while checked exceptions require explicit rules unless configured differently.

EXAMPLE / MAPPING OR QUERY

```
@Transactional(rollbackFor=PaymentFailure.class)
```

TRACE THE IDEA



WATCH FOR

Catching a failure and returning normally may commit partial work.

FOLLOW-UP QUESTION

How do you roll back for a checked business exception?

ANSWER

Declare rollbackFor or use a suitable domain runtime exception after examining the service contract.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TRANSACTION BOUNDARIES AND SPRING INTEGRATION

Q054
What does readOnly=true mean?

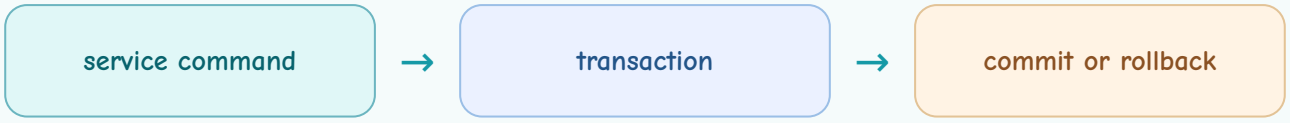
DIRECT ANSWER

It is a transaction hint that can reduce unnecessary work for reads, but enforcement and optimizations depend on provider and database.

EXAMPLE / MAPPING OR QUERY

```
@Transactional(readOnly=true) public List<OrderDto> list()
```

TRACE THE IDEA



WATCH FOR

Read-only does not replace database privileges.

FOLLOW-UP QUESTION

Does it guarantee write attempts are rejected?

ANSWER

No. Do not treat it as a security or integrity boundary.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TRANSACTION BOUNDARIES AND SPRING INTEGRATION

Q055
Why can transaction self-invocation fail?

DIRECT ANSWER

A call from one method of a Spring bean to another on the same instance bypasses the default proxy interception.

EXAMPLE / MAPPING OR QUERY

```
service.process(); // through Spring proxy
```

TRACE THE IDEA



WATCH FOR

Adding annotations without testing boundaries can silently fail.

FOLLOW-UP QUESTION

How can you fix it?

ANSWER

Move the transactional operation to another Spring-managed collaborator, or use a deliberately selected interception strategy.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

12 / CHAPTER 12

Concurrency, isolation and locking

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q056 What does optimistic locking with @Version do?

Q057 What is pessimistic locking?

Q058 Why can two updates lose one writer's changes?

Q059 What is write skew and why is it dangerous?

Q060 How should deadlock retries work?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / CONCURRENCY, ISOLATION AND LOCKING

Q056
What does optimistic locking with @Version do?

DIRECT ANSWER

Hibernate includes a version condition in managed entity updates/deletes, detecting some concurrent modifications.

EXAMPLE / MAPPING OR QUERY

```
@Version long version;
```

TRACE THE IDEA



WATCH FOR

Version checks cover updates that participate in the protocol.

FOLLOW-UP QUESTION

How should a stale update be handled?

ANSWER

Return a conflict or retry the complete decision using fresh state when the operation is safely retryable.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CONCURRENCY, ISOLATION AND LOCKING

Q057
What is pessimistic locking?

DIRECT ANSWER

A database lock is acquired to coordinate access before a decision, potentially blocking competing transactions.

EXAMPLE / MAPPING OR QUERY

```
em.find(Stock.class,id,LockModeType.PESSIMISTIC_WRITE);
```

TRACE THE IDEA



WATCH FOR

A long remote call while holding a row lock harms throughput.

FOLLOW-UP QUESTION

What costs does it add?

ANSWER

Waiting, deadlocks, connection occupancy and timeout management.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CONCURRENCY, ISOLATION AND LOCKING

Q058

Why can two updates lose one writer's changes?

DIRECT ANSWER

Both transactions can read the same old state and write a new value without an adequate conflict mechanism.

EXAMPLE / MAPPING OR QUERY

```
update stock set qty=qty-1 where id=? and qty>0
```

TRACE THE IDEA



WATCH FOR

A Java synchronized block does not coordinate multiple JVMs.

FOLLOW-UP QUESTION

Can @Version fix every race?

ANSWER

No. Cross-row invariants and bulk/native writes need their own constraints and transaction design.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CONCURRENCY, ISOLATION AND LOCKING

Q059
What is write skew and why is it dangerous?

DIRECT ANSWER

Two transactions can each update different rows based on a shared condition that becomes false after both commits.

EXAMPLE / MAPPING OR QUERY

```
two doctors each disable own on-call row
```

TRACE THE IDEA



WATCH FOR

Row-level @Version on separate rows alone may not prevent it.

FOLLOW-UP QUESTION

How can you protect the invariant?

ANSWER

Use suitable serializable isolation or explicit locking/coordination around the complete invariant.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / CONCURRENCY, ISOLATION AND LOCKING

Q060
How should deadlock retries work?

DIRECT ANSWER

Retry the complete idempotent transaction with a bounded policy and fresh reads after the database aborts one participant.

EXAMPLE / MAPPING OR QUERY

```
retry with jitter and maxAttempts=3
```

TRACE THE IDEA



WATCH FOR

Blind retries can repeat external side effects.

FOLLOW-UP QUESTION

Why is retrying only the last SQL statement unsafe?

ANSWER

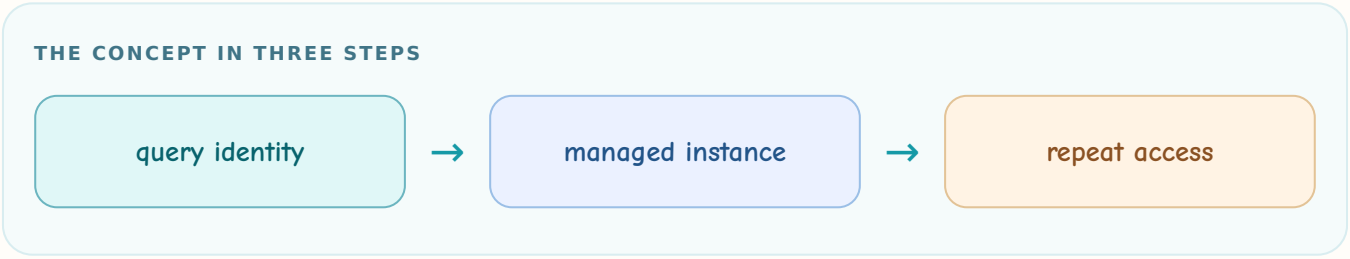
Previous reads and decisions may no longer be valid; the entire transaction must be re-evaluated.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

13 / CHAPTER 13

First-level cache and persistence context

Learn the contract, trace an example, then explain where the guarantee ends.



Questions in this chapter

Q061 What is Hibernate's first-level cache?

Q062 Does the first-level cache guarantee fresh database values?

Q063 How can a persistence context grow too large?

Q064 What does EntityManager.contains(entity) tell you?

Q065 Why should entities not be used as cross-request cache objects?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / FIRST-LEVEL CACHE AND PERSISTENCE CONTEXT

Q061
What is Hibernate's first-level cache?

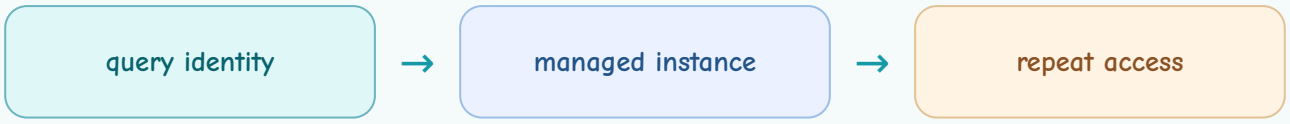
DIRECT ANSWER

It is the identity map of the persistence context: the same entity key maps to one managed object within that context.

EXAMPLE / MAPPING OR QUERY

```
em.find(User.class,1L) == em.find(User.class,1L)
```

TRACE THE IDEA



WATCH FOR

A first-level cache is not shared across requests.

FOLLOW-UP QUESTION

Will repeated find() always run SQL?

ANSWER

Not if the entity is already managed and no refresh semantics force a read.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FIRST-LEVEL CACHE AND PERSISTENCE CONTEXT

Q062

Does the first-level cache guarantee fresh database values?

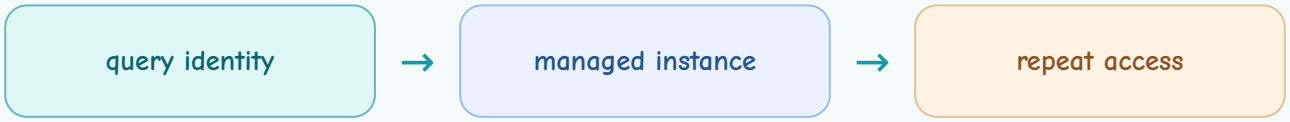
DIRECT ANSWER

No. Managed state can become stale when another transaction or bulk statement changes rows.

EXAMPLE / MAPPING OR QUERY

```
em.refresh(customer);
```

TRACE THE IDEA



WATCH FOR

A cached entity is not a substitute for locking.

FOLLOW-UP QUESTION

How do you reload authoritative state?

ANSWER

Use refresh, clear, or a new persistence context as the correctness scenario requires.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FIRST-LEVEL CACHE AND PERSISTENCE CONTEXT

Q063
How can a persistence context grow too large?

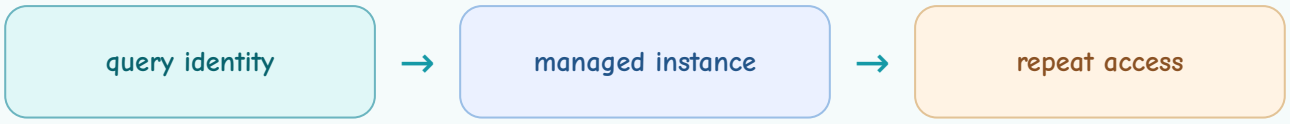
DIRECT ANSWER

Loading or persisting many entities keeps managed references and snapshots until they are detached or cleared.

EXAMPLE / MAPPING OR QUERY

```
if(i%100==0){ em.flush(); em.clear(); }
```

TRACE THE IDEA



WATCH FOR

Unbounded persistence contexts can exhaust heap.

FOLLOW-UP QUESTION

How do you batch large imports?

ANSWER

Process bounded chunks, flush and clear deliberately, and avoid materializing the entire source.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FIRST-LEVEL CACHE AND PERSISTENCE CONTEXT

Q064

What does EntityManager.contains(entity) tell you?

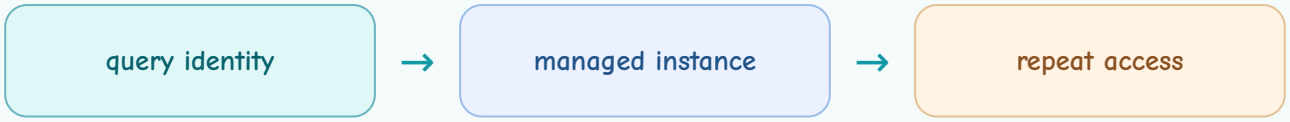
DIRECT ANSWER

It reports whether a specific entity instance is managed by that EntityManager.

EXAMPLE / MAPPING OR QUERY

```
boolean tracked=em.contains(order);
```

TRACE THE IDEA



WATCH FOR

A removed entity may be managed until context transitions complete.

FOLLOW-UP QUESTION

Does contains() check that a database row exists?

ANSWER

No. It reports management state, not committed existence.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / FIRST-LEVEL CACHE AND PERSISTENCE CONTEXT

Q065

Why should entities not be used as cross-request cache objects?

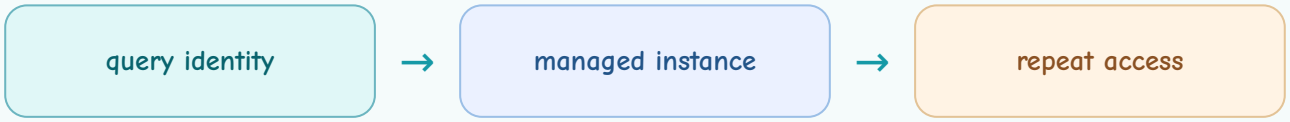
DIRECT ANSWER

They can carry proxies, stale state, context-dependent semantics and mutable relationships tied to a transaction.

EXAMPLE / MAPPING OR QUERY

```
Cache<OrderDto> not Cache<OrderEntity>
```

TRACE THE IDEA



WATCH FOR

Do not serialize an open persistence context across requests.

FOLLOW-UP QUESTION

What should cross-request caches store?

ANSWER

Immutable IDs or deliberately versioned DTO/read models with a clear freshness policy.

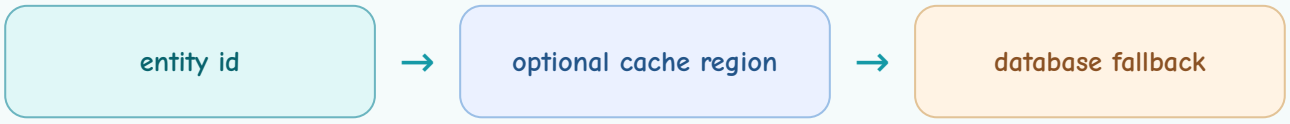
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

14 / CHAPTER 14

Second-level and query caches

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q066 What is the second-level cache?

Q067 How is the second-level cache different from first-level cache?

Q068 What are READ_ONLY and READ_WRITE cache concurrency strategies?

Q069 What does a query cache store?

Q070 How should a distributed cache react to external writers?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / SECOND-LEVEL AND QUERY CACHES

Q066
What is the second-level cache?

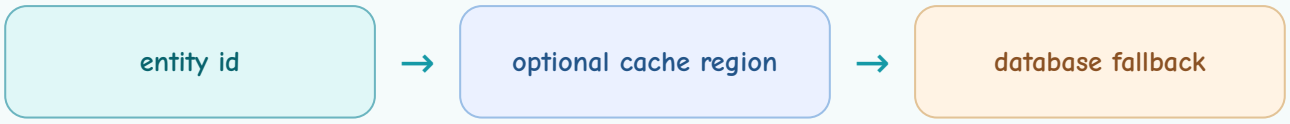
DIRECT ANSWER

It is an optional SessionFactory-scoped cache for eligible entity/collection state, managed through configured cache regions/providers.

EXAMPLE / MAPPING OR QUERY

```
@Cache(usage = READ_ONLY) on immutable reference entity
```

TRACE THE IDEA



WATCH FOR

A cache provider is required; an annotation alone is insufficient.

FOLLOW-UP QUESTION

Is it enabled and populated for all entities by default?

ANSWER

No. Configure a suitable region provider and selective caching for entities where it is safe.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SECOND-LEVEL AND QUERY CACHES

Q067

How is the second-level cache different from first-level cache?

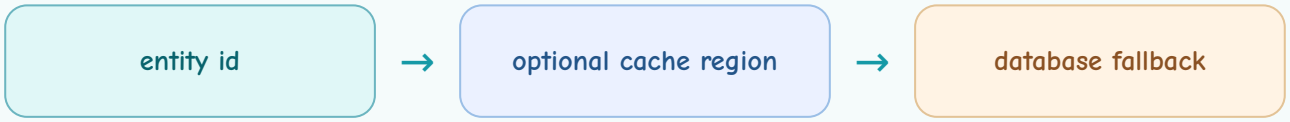
DIRECT ANSWER

First-level cache is mandatory per persistence context; second-level cache is optional across sessions and adds concurrency/freshness policies.

EXAMPLE / MAPPING OR QUERY

```
Session A -> L2 region <- Session B
```

TRACE THE IDEA



WATCH FOR

L2 cache does not make entity instances thread-safe.

FOLLOW-UP QUESTION

Can two sessions share the same managed Java object?

ANSWER

No. They can reuse cached state but each manages its own entity instance.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SECOND-LEVEL AND QUERY CACHES

Q068

What are READ_ONLY and READ_WRITE cache concurrency strategies?

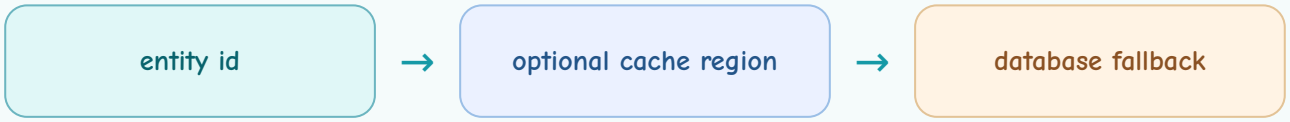
DIRECT ANSWER

READ_ONLY suits immutable cached data; READ_WRITE uses provider coordination for mutable entity state under supported assumptions.

EXAMPLE / MAPPING OR QUERY

```
@Cache(usage=CacheConcurrencyStrategy.READ_WRITE)
```

TRACE THE IDEA



WATCH FOR

Choose strategies by data mutation and consistency needs.

FOLLOW-UP QUESTION

Does READ_WRITE remove database transactions?

ANSWER

No. Business correctness still depends on database guarantees and cache configuration.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SECOND-LEVEL AND QUERY CACHES

Q069
What does a query cache store?

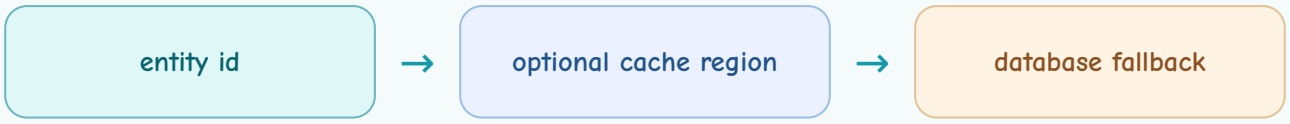
DIRECT ANSWER

It can cache query-result identifiers/scalars for eligible cacheable queries, often relying on entity cache/invalidations as well.

EXAMPLE / MAPPING OR QUERY

```
query.setHint("org.hibernate.cacheable",true);
```

TRACE THE IDEA



WATCH FOR

A query result cache is not a universal SQL performance fix.

FOLLOW-UP QUESTION

Should you enable it globally?

ANSWER

No. Measure hit rate and invalidation behavior; frequently changing tables may yield poor value.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SECOND-LEVEL AND QUERY CACHES

Q070

How should a distributed cache react to external writers?

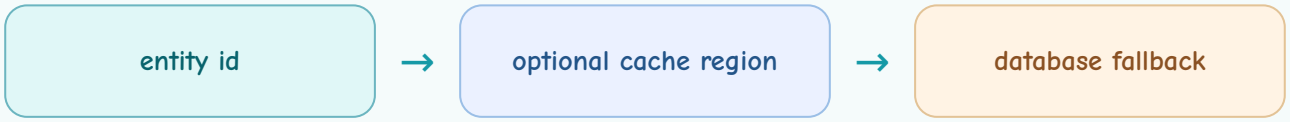
DIRECT ANSWER

Design eviction/refresh when other applications update the same tables, or avoid caching data whose updates cannot be coordinated.

EXAMPLE / MAPPING OR QUERY

```
external ETL UPDATE -> evict affected cache
```

TRACE THE IDEA



WATCH FOR

Stale cross-service cache state can violate business expectations.

FOLLOW-UP QUESTION

Can Hibernate know every external SQL write automatically?

ANSWER

No. Out-of-band changes may not trigger provider cache invalidation.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

15 / CHAPTER 15

Batching, IDs and write optimization

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q071 What is JDBC statement batching in Hibernate?

Q072 Why can IDENTITY IDs reduce insert batching?

Q073 What is an allocation size for sequences?

Q074 How do ordered inserts and updates help?

Q075 How can you process a million rows safely?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / BATCHING, IDS AND WRITE OPTIMIZATION

Q071
What is JDBC statement batching in Hibernate?

DIRECT ANSWER

Hibernate can group compatible INSERT/UPDATE/DELETE statements for fewer database round trips when configured and supported.

EXAMPLE / MAPPING OR QUERY

```
hibernate.jdbc.batch_size=32
```

TRACE THE IDEA



WATCH FOR

Batch size is a throughput tuning parameter, not an invariant.

FOLLOW-UP QUESTION

Will batching always improve performance?

ANSWER

Not automatically. Benchmark with your driver, generator strategy, table and batch size.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BATCHING, IDS AND WRITE OPTIMIZATION

Q072
Why can IDENTITY IDs reduce insert batching?

DIRECT ANSWER

The generated identifier may require immediate insertion to retrieve its value, limiting how Hibernate groups inserts.

EXAMPLE / MAPPING OR QUERY

```
@GeneratedValue(strategy=SEQUENCE)
```

TRACE THE IDEA

pending inserts → JDBC batches → fewer network trips

WATCH FOR

Changing ID strategy needs migration and ordering review.

FOLLOW-UP QUESTION

What is a possible alternative?

ANSWER

Use a suitable sequence-based strategy on databases supporting it, considering allocation and identity requirements.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BATCHING, IDS AND WRITE OPTIMIZATION

Q073
What is an allocation size for sequences?

DIRECT ANSWER

It lets identifiers be reserved in blocks to reduce calls to a sequence provider, depending on optimizer behavior.

EXAMPLE / MAPPING OR QUERY

```
@SequenceGenerator(allocationSize=50)
```

TRACE THE IDEA

pending inserts → JDBC batches → fewer network trips

WATCH FOR

Use a separate business rule if contiguous invoice numbering is required.

FOLLOW-UP QUESTION

Will unused IDs after a crash be a bug?

ANSWER

Gaps are normal in many generated ID strategies; never assume sequential gap-free business numbers.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BATCHING, IDS AND WRITE OPTIMIZATION

Q074
How do ordered inserts and updates help?

DIRECT ANSWER

Grouping statements by entity and key can make JDBC batching more effective, though it introduces ordering and processing cost.

EXAMPLE / MAPPING OR QUERY

```
hibernate.order_inserts=true
```

TRACE THE IDEA



WATCH FOR

Reordered statements can expose previously hidden constraint timing.

FOLLOW-UP QUESTION

Should these flags always be enabled?

ANSWER

Measure realistic workloads and verify any constraint-sensitive operations.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BATCHING, IDS AND WRITE OPTIMIZATION

Q075

How can you process a million rows safely?

DIRECT ANSWER

Prefer chunked reads or streaming with bounded state, per-chunk transactions, and flush/clear instead of loading all entities.

EXAMPLE / MAPPING OR QUERY

```
for each batch -> write -> flush -> clear
```

TRACE THE IDEA



WATCH FOR

Bulk tooling changes guarantees; read the API contract.

FOLLOW-UP QUESTION

When might StatelessSession be useful?

ANSWER

For specialized bulk operations where you deliberately give up normal persistence-context behavior and some event/cascade semantics.

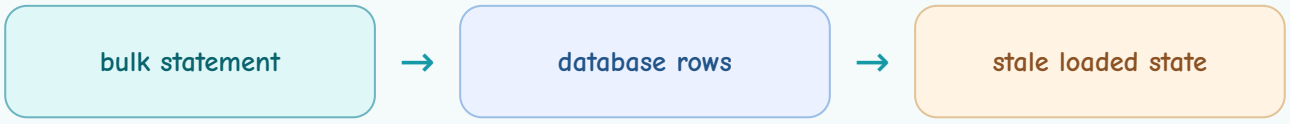
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

16 / CHAPTER 16

Bulk updates, native SQL and consistency

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q076 How does JPQL bulk UPDATE differ from entity dirty checking?

Q077 When should you use a bulk delete?

Q078 How do you combine native SQL with managed entities safely?

Q079 Why should query update counts be checked?

Q080 How do you avoid long retention cleanup transactions?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / BULK UPDATES, NATIVE SQL AND CONSISTENCY

Q076

How does JPQL bulk UPDATE differ from entity dirty checking?

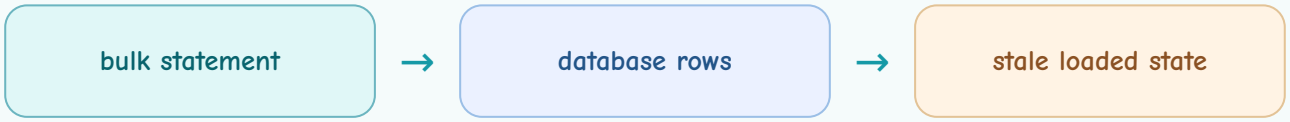
DIRECT ANSWER

Bulk DML acts directly on matching database rows and bypasses normal per-entity dirty checking and lifecycle callbacks.

EXAMPLE / MAPPING OR QUERY

```
update Order o set o.status=:s where o.id in :ids
```

TRACE THE IDEA



WATCH FOR

Bulk DML may bypass optimistic-version checks unless explicitly designed.

FOLLOW-UP QUESTION

What happens to already-managed entities?

ANSWER

They may remain stale; clear or refresh the context after the bulk operation.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BULK UPDATES, NATIVE SQL AND CONSISTENCY

Q077
When should you use a bulk delete?

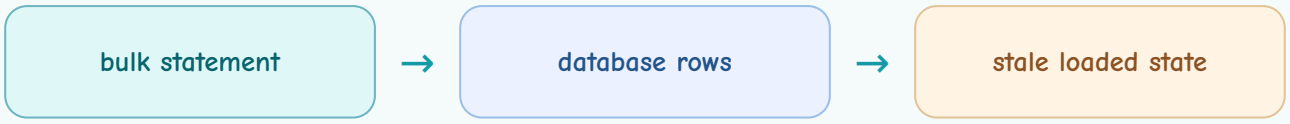
DIRECT ANSWER

Use it for validated set-based removal with correct tenant, retention, FK and audit rules, rather than loading every row.

EXAMPLE / MAPPING OR QUERY

```
delete from ExpiredToken t where t.expiresAt<:now
```

TRACE THE IDEA



WATCH FOR

Not all object lifecycle hooks run in bulk SQL.

FOLLOW-UP QUESTION

Does it invoke entity @PreRemove callbacks?

ANSWER

Bulk DML generally bypasses entity callbacks/cascades; validate requirements first.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BULK UPDATES, NATIVE SQL AND CONSISTENCY

Q078

How do you combine native SQL with managed entities safely?

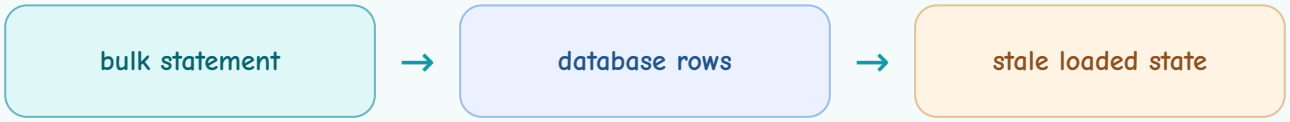
DIRECT ANSWER

Flush pending relevant changes, execute bounded native SQL, and clear/refresh affected managed state as needed.

EXAMPLE / MAPPING OR QUERY

```
em.flush(); em.createNativeQuery("update ...").executeUpdate(); em.clear();
```

TRACE THE IDEA



WATCH FOR

Stale context state can overwrite SQL changes later.

FOLLOW-UP QUESTION

Can a native UPDATE invalidate every cache?

ANSWER

Do not assume complete invalidation for arbitrary external statements.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BULK UPDATES, NATIVE SQL AND CONSISTENCY

Q079
Why should query update counts be checked?

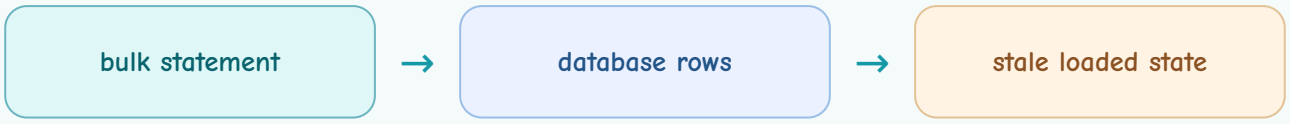
DIRECT ANSWER

Affected-row counts can express optimistic concurrency, absent rows or insufficient-stock outcomes.

EXAMPLE / MAPPING OR QUERY

```
UPDATE inventory SET qty=qty-1 WHERE qty>0 AND id=:id
```

TRACE THE IDEA



WATCH FOR

An UPDATE that returns zero may be an expected conflict.

FOLLOW-UP QUESTION

What should zero changed rows mean?

ANSWER

Define a business response; distinguish missing identity, stale version or predicate failure where needed.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / BULK UPDATES, NATIVE SQL AND CONSISTENCY

Q080

How do you avoid long retention cleanup transactions?

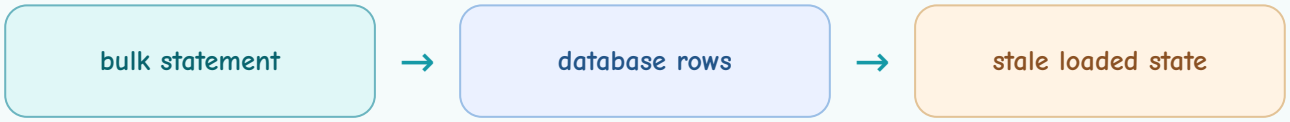
DIRECT ANSWER

Delete in limited batches, use indexed predicates and monitor locks, replication lag and vacuum/purge behavior.

EXAMPLE / MAPPING OR QUERY

```
DELETE FROM events WHERE created_at < :cutoff LIMIT ...
```

TRACE THE IDEA



WATCH FOR

DELETE LIMIT syntax and semantics differ across databases.

FOLLOW-UP QUESTION

Should ORM entity removal be used for millions of old rows?

ANSWER

Usually consider reviewed bulk operations or archival jobs instead of one giant persistence context.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

17 / CHAPTER 17

Inheritance, polymorphism and mapping tradeoffs

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q081 What is SINGLE_TABLE inheritance?

Q082 What is JOINED inheritance?

Q083 What is TABLE_PER_CLASS inheritance?

Q084 How do @MappedSuperclass and @Entity inheritance differ?

Q085 How would you choose an inheritance strategy?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / INHERITANCE, POLYMORPHISM AND MAPPING TRADEOFFS

Q081
What is SINGLE_TABLE inheritance?

DIRECT ANSWER

Several related entity subclasses live in one table with a discriminator, avoiding joins but allowing nullable subtype columns.

EXAMPLE / MAPPING OR QUERY

```
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
```

TRACE THE IDEA



WATCH FOR

Database constraints must still reflect subtype rules.

FOLLOW-UP QUESTION

When is it unsuitable?

ANSWER

When many subtype-specific columns and constraints make the common table unwieldy.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / INHERITANCE, POLYMORPHISM AND MAPPING TRADEOFFS

Q082
What is JOINED inheritance?

DIRECT ANSWER

Common fields live in a base table and subtype fields in separate tables linked by keys.

EXAMPLE / MAPPING OR QUERY

```
@Inheritance(strategy=InheritanceType.JOINED)
```

TRACE THE IDEA



WATCH FOR

Evaluate query patterns rather than choosing solely for normalization.

FOLLOW-UP QUESTION

What query cost can it add?

ANSWER

Polymorphic reads may require joins, making SQL more complex.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / INHERITANCE, POLYMORPHISM AND MAPPING TRADEOFFS

Q083
What is TABLE_PER_CLASS inheritance?

DIRECT ANSWER

Concrete classes have independent tables; polymorphic queries may need unions.

EXAMPLE / MAPPING OR QUERY

```
@Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)
```

TRACE THE IDEA



WATCH FOR

Identifier strategy and query support need validation.

FOLLOW-UP QUESTION

Where does this become expensive?

ANSWER

Querying a base type across many subclasses can require UNION-style work.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / INHERITANCE, POLYMORPHISM AND MAPPING TRADEOFFS

Q084

How do @MappedSuperclass and @Entity inheritance differ?

DIRECT ANSWER

A mapped superclass contributes mapping fields but is not itself an entity you query polymorphically.

EXAMPLE / MAPPING OR QUERY

```
@MappedSuperclass class Audited { Instant createdAt; }
```

TRACE THE IDEA



WATCH FOR

Do not query a mapped superclass as though it owns a table.

FOLLOW-UP QUESTION

When does @MappedSuperclass fit?

ANSWER

For shared audit fields when no domain polymorphism is required.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / INHERITANCE, POLYMORPHISM AND MAPPING TRADEOFFS

Q085

How would you choose an inheritance strategy?

DIRECT ANSWER

Start from domain substitutions, query patterns, schema constraints and anticipated subtype growth.

EXAMPLE / MAPPING OR QUERY

```
PaymentMethod -> CardDetails / BankDetails
```

TRACE THE IDEA



WATCH FOR

Do not make inheritance a default just because Java supports extends.

FOLLOW-UP QUESTION

Can composition be better?

ANSWER

Yes. A status or separate value component may be simpler than a deep entity hierarchy.

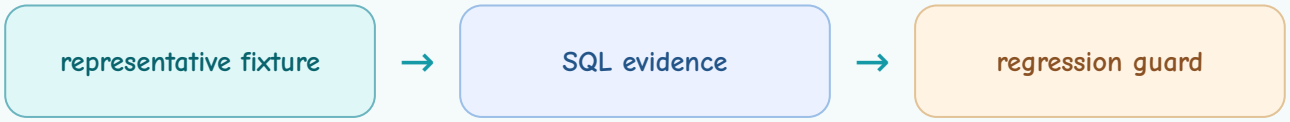
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

18 / CHAPTER 18

Testing, schema migrations and verification

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q086 What is a useful Hibernate integration test?

Q087 Why should tests flush and clear before reading back?

Q088 Should Hibernate hbm2ddl auto-update production schemas?

Q089 Why can H2 tests differ from PostgreSQL or MySQL?

Q090 How can you test SQL query counts?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / TESTING, SCHEMA MIGRATIONS AND VERIFICATION

Q086
What is a useful Hibernate integration test?

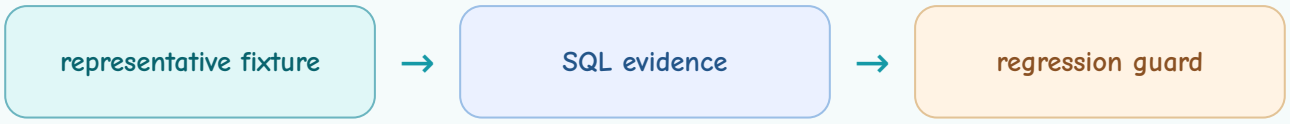
DIRECT ANSWER

It uses a real mapped EntityManager and database schema to verify flush, SQL constraints, associations, transactions and queries.

EXAMPLE / MAPPING OR QUERY

```
persist -> flush -> clear -> find -> assert
```

TRACE THE IDEA



WATCH FOR

A passing service mock test does not validate ORM mapping.

FOLLOW-UP QUESTION

Why aren't mocks sufficient?

ANSWER

Mocks don't prove actual SQL, joins, flush timing or constraint behavior.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TESTING, SCHEMA MIGRATIONS AND VERIFICATION

Q087

Why should tests flush and clear before reading back?

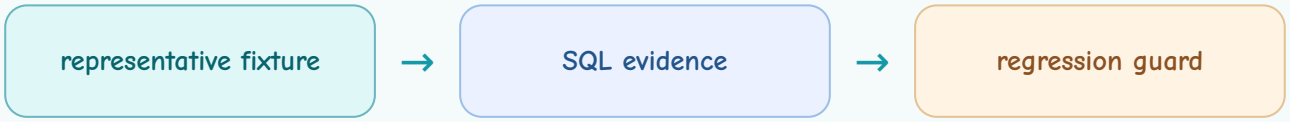
DIRECT ANSWER

It prevents first-level cache state from hiding a missing database write or incorrect mapping.

EXAMPLE / MAPPING OR QUERY

```
em.flush(); em.clear(); assertEquals(..., em.find(...));
```

TRACE THE IDEA



WATCH FOR

Always distinguish persistence-context behavior from DB evidence.

FOLLOW-UP QUESTION

What does an immediate find() without clear prove?

ANSWER

It may only return the managed object already in memory.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TESTING, SCHEMA MIGRATIONS AND VERIFICATION

Q088

Should Hibernate hbm2ddl auto-update production schemas?

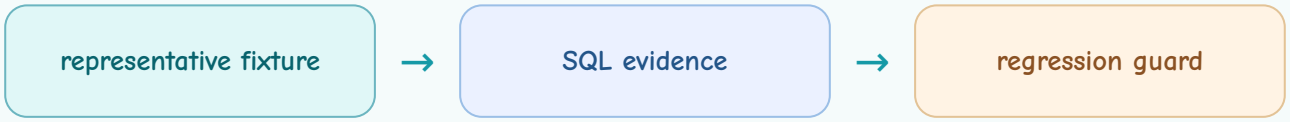
DIRECT ANSWER

Production schema changes should be version-controlled and reviewed using migration tools such as Flyway or Liquibase.

EXAMPLE / MAPPING OR QUERY

```
spring.jpa.hibernate.ddl-auto=validate
```

TRACE THE IDEA



WATCH FOR

Schema generation is not a production migration plan.

FOLLOW-UP QUESTION

What should hbm2ddl use in production?

ANSWER

Typically validation at most, depending on deployment policy; never rely on uncontrolled auto-migrations.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TESTING, SCHEMA MIGRATIONS AND VERIFICATION

Q089

Why can H2 tests differ from PostgreSQL or MySQL?

DIRECT ANSWER

SQL dialects differ in constraints, types, locking, sequences and execution plans, even when a simple test passes.

EXAMPLE / MAPPING OR QUERY

```
Testcontainers with PostgreSQL in CI
```

TRACE THE IDEA

representative fixture → SQL evidence → regression guard

WATCH FOR

H2 cannot prove production deadlock or isolation behavior.

FOLLOW-UP QUESTION

What additional test is valuable?

ANSWER

Run important integration tests against the actual database family and version, often through containers.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / TESTING, SCHEMA MIGRATIONS AND VERIFICATION

Q090
How can you test SQL query counts?

DIRECT ANSWER

Enable Hibernate Statistics or instrument JDBC, seed enough entities to expose N+1, and assert bounded statements.

EXAMPLE / MAPPING OR QUERY

```
statistics.getPrepareStatementCount()
```

TRACE THE IDEA

representative fixture → SQL evidence → regression guard

WATCH FOR

Query-count assertions should be stable under test setup.

FOLLOW-UP QUESTION

Why isn't one-row test data enough?

ANSWER

N+1 and multiplication problems appear as the number of parents/children grows.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

19 / CHAPTER 19

Observability and production performance

Learn the contract, trace an example, then explain where the guarantee ends.



Questions in this chapter

Q091 Which Hibernate logs and metrics matter first?

Q092 How do you investigate slow Hibernate requests?

Q093 What is connection-pool exhaustion?

Q094 Why is a cartesian product dangerous in ORM queries?

Q095 When does the second-level cache make performance worse?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / OBSERVABILITY AND PRODUCTION PERFORMANCE

Q091
Which Hibernate logs and metrics matter first?

DIRECT ANSWER

Inspect SQL statements, bind values safely in nonproduction, query counts, slow statements, transaction duration and connection-pool usage.

EXAMPLE / MAPPING OR QUERY

```
hibernate.generate_statistics=true // targeted diagnostics
```

TRACE THE IDEA



WATCH FOR

SQL logging can leak personal data and passwords.

FOLLOW-UP QUESTION

Should you log every bind parameter in production?

ANSWER

No. Sensitive values and overhead need controls and redaction.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / OBSERVABILITY AND PRODUCTION PERFORMANCE

Q092

How do you investigate slow Hibernate requests?

DIRECT ANSWER

Trace endpoint latency through pool wait, ORM queries and database execution plans, then fix the measured bottleneck.

EXAMPLE / MAPPING OR QUERY

```
EXPLAIN (ANALYZE, BUFFERS) ... // Postgres
```

TRACE THE IDEA



WATCH FOR

Do not optimize only Java CPU when SQL dominates latency.

FOLLOW-UP QUESTION

Can a second-level cache solve a missing index?

ANSWER

It may hide some reads, but doesn't remove write cost or uncached query problems.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / OBSERVABILITY AND PRODUCTION PERFORMANCE

Q093
What is connection-pool exhaustion?

DIRECT ANSWER

Active connections occupy the configured pool so new operations wait or time out, often because transactions are slow or leaked.

EXAMPLE / MAPPING OR QUERY

```
Hikari maximumPoolSize vs DB max_connections
```

TRACE THE IDEA

ORM operations → database metrics → root cause

WATCH FOR

More pool connections can overload the database.

FOLLOW-UP QUESTION

What should you inspect?

ANSWER

Pool active/idle/pending, slow SQL, lock waits and remote calls inside transactions.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / OBSERVABILITY AND PRODUCTION PERFORMANCE

Q094

Why is a cartesian product dangerous in ORM queries?

DIRECT ANSWER

Joining several to-many paths can multiply rows and inflate transfer, materialization and memory costs.

EXAMPLE / MAPPING OR QUERY

```
Order(3 items) JOIN payments(4) => 12 rows
```

TRACE THE IDEA



WATCH FOR

DISTINCT doesn't undo all wasted database work.

FOLLOW-UP QUESTION

How do you fix it?

ANSWER

Fetch bounded relationships separately, project DTOs or aggregate at the correct grain.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / OBSERVABILITY AND PRODUCTION PERFORMANCE

Q095

When does the second-level cache make performance worse?

DIRECT ANSWER

Poor hit rates, frequent invalidations, extra serialization or coherence work can outweigh avoided queries.

EXAMPLE / MAPPING OR QUERY

```
cache hit %, region size, query p99
```

TRACE THE IDEA



WATCH FOR

Cache success must be judged by correctness as well as speed.

FOLLOW-UP QUESTION

How would you decide to enable it?

ANSWER

Measure hit/miss rate, invalidations, stale-read tolerance and memory cost under realistic load.

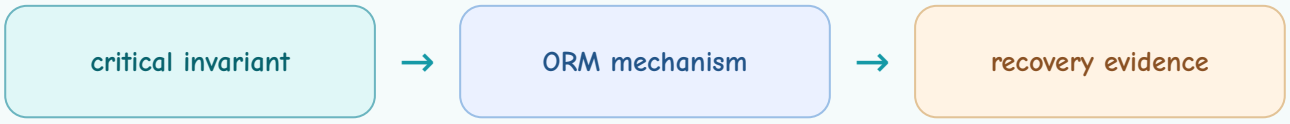
Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

20 / CHAPTER 20

Senior architect scenarios and upgrades

Learn the contract, trace an example, then explain where the guarantee ends.

THE CONCEPT IN THREE STEPS



Questions in this chapter

Q096 How would you design an order service with Hibernate?

Q097 How do you prevent double booking through JPA?

Q098 How would you migrate Hibernate 5 to Hibernate 6 safely?

Q099 How do you handle multi-tenancy with Hibernate?

Q100 What does a strong ORM architecture interview answer include?

A good answer explains not only what Hibernate does, but also **what the database must still guarantee**.

INTERVIEW QUESTION / SENIOR ARCHITECT SCENARIOS AND UPGRADES

Q096

How would you design an order service with Hibernate?

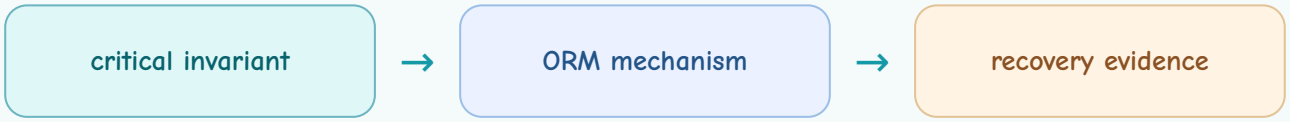
DIRECT ANSWER

Use a small transactional aggregate for order, items and durable idempotency state; validate stock with an authoritative conditional write.

EXAMPLE / MAPPING OR QUERY

```
Order + Items + Outbox -> local COMMIT
```

TRACE THE IDEA



WATCH FOR

A @Transactional method does not provide distributed atomicity.

FOLLOW-UP QUESTION

What part cannot be in the same DB transaction?

ANSWER

An external payment network; represent it as a recoverable workflow with outbox and reconciliation.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SENIOR ARCHITECT SCENARIOS AND UPGRADES

Q097

How do you prevent double booking through JPA?

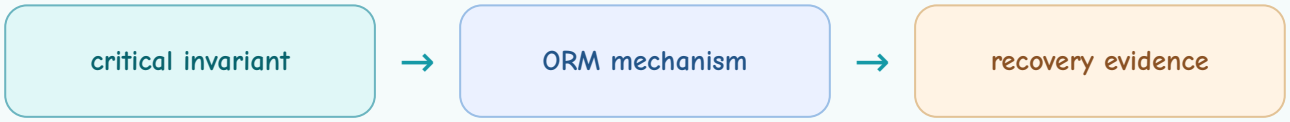
DIRECT ANSWER

Use a database uniqueness/conditional update rule and an idempotency key under a transaction, rather than read-then-insert checks.

EXAMPLE / MAPPING OR QUERY

```
UNIQUE(event_id, seat_id) on confirmed allocation
```

TRACE THE IDEA



WATCH FOR

Two JVMs can race past a Java precheck.

FOLLOW-UP QUESTION

Can optimistic locking alone enforce seat uniqueness?

ANSWER

Only if all conflicting decisions touch the same protected versioned state; database uniqueness is often safer.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SENIOR ARCHITECT SCENARIOS AND UPGRADES

Q098

How would you migrate Hibernate 5 to Hibernate 6 safely?

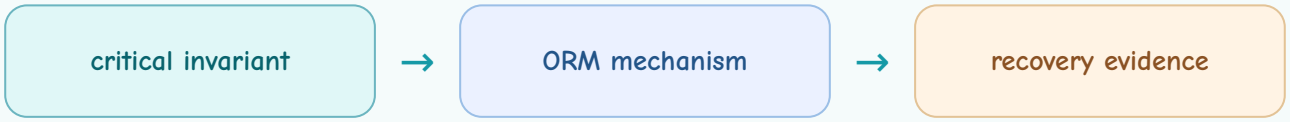
DIRECT ANSWER

Inventory javax-to-jakarta namespaces, changed query/mapping behavior, dialects and extensions, then upgrade libraries and run integration tests.

EXAMPLE / MAPPING OR QUERY

```
javax.persistence.* -> jakarta.persistence.*
```

TRACE THE IDEA



WATCH FOR

Changing imports alone is not a complete migration.

FOLLOW-UP QUESTION

What should block rollout?

ANSWER

Schema/query regressions, unsupported APIs and unverified SQL/transaction behavior.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SENIOR ARCHITECT SCENARIOS AND UPGRADES

Q099

How do you handle multi-tenancy with Hibernate?

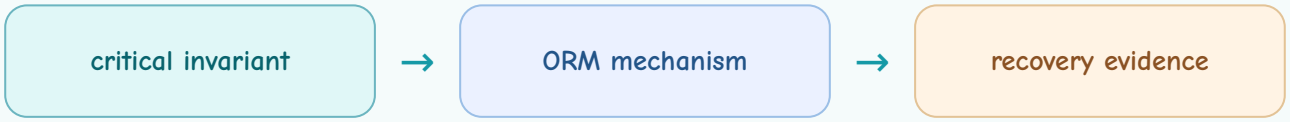
DIRECT ANSWER

Choose a deliberate strategy such as database/schema separation or discriminator-based scoping, with tenant identity from trusted security context.

EXAMPLE / MAPPING OR QUERY

```
tenant context -> resolver -> DB routing/filter
```

TRACE THE IDEA



WATCH FOR

Tenant filtering must never depend solely on user input.

FOLLOW-UP QUESTION

What are high-risk bypass paths?

ANSWER

Native SQL, bulk DML, caches, batch jobs and administrative endpoints can miss tenant constraints.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

INTERVIEW QUESTION / SENIOR ARCHITECT SCENARIOS AND UPGRADES

Q100

What does a strong ORM architecture interview answer include?

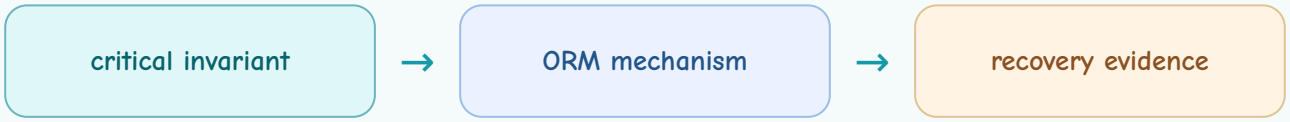
DIRECT ANSWER

State the business invariant, mapping and ownership, fetch/query plan, transaction boundary, failure recovery and observable evidence.

EXAMPLE / MAPPING OR QUERY

```
requirement -> invariant -> SQL evidence -> recovery
```

TRACE THE IDEA



WATCH FOR

Naming annotations is not enough to demonstrate correctness.

FOLLOW-UP QUESTION

How would you prove the system meets its guarantee?

ANSWER

Use constraint tests, concurrency tests, representative data, query plans and failure-injection checks.

Remember: Explain the behavior, the SQL/transaction effect, and one failure case.

CODING LABS

12 practical Hibernate/JPA labs

Each source scenario is included in the Maven project and marked for local execution.

Run the complete lab project

cd Hibernate_ORM_Interview_Labs

mvn -q compile exec:java

Expected goal: 12 PASS messages. Actual integration run not yet verified.

C01 Persist and read an entity

Load from a new EntityManager

C02 First-level cache identity

Same managed instance per context

C03 Dirty checking

Change and commit a managed field

C04 Detach and merge

Use the returned managed copy

C05 Flush versus commit

Rollback an already flushed update

C06 Cascade and orphan removal

Create and remove an owned child

C07 JPQL scalar projection

Bind a parameter and select status

C08 Bounded join fetching

Load one order with customer and lines

C09 Optimistic locking

Detect and reject a stale writer

C10 Bulk JPQL and stale state

Refresh after a direct bulk update

C11 Conditional stock update

Only one stock decrement succeeds

C12 Detach and tracking

A detached mutation is not committed

HANDS-ON LAB C01

Persist and read an entity

An entity inserted in one transaction must be retrievable from another transaction.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab01(){
  long id=customer();String name=tx(em->em.find(CustomerEntity.class,id).getName());
  check(name.startsWith("Customer"),"C01 persist/find");
 }
```

Expected check

Run the Maven project locally and confirm **C01 PASS**, then change the input to test a failure case.

Explain it in an interview

Use an actual second EntityManager to avoid mistaking cache state for persisted state.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C02

First-level cache identity

Two finds of the same ID inside one context return the same managed instance.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab02(){
  long id=customer();tx(em->{CustomerEntity a=em.find(CustomerEntity.class,id);CustomerEntity
b=em.find(CustomerEntity.class,id);
   check(a==b,"C02 first-level identity");return null;});
 }
```

Expected check

Run the Maven project locally and confirm **C02 PASS**, then change the input to test a failure case.

Explain it in an interview

Contrast this with a new context; do not claim global shared identity.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C03

Dirty checking

Change a managed field, commit, and read the change in a fresh context.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab03(){
  long id=customer();tx(em->{em.find(CustomerEntity.class,id).setName("Updated");return null;});
  check(tx(em->em.find(CustomerEntity.class,id).getName()).equals("Updated"),"C03 dirty
checking");
 }
```

Expected check

Run the Maven project locally and confirm **C03 PASS**, then change the input to test a failure case.

Explain it in an interview

Contrast with detached state; note flush timing.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C04

Detach and merge

Modify a detached object then merge and use the returned managed instance.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab04(){
  long id=customer();CustomerEntity detached=tx(em->{CustomerEntity
c=em.find(CustomerEntity.class,id);em.detach(c);return c;});
  detached.setName("Merged");tx(em->{CustomerEntity m=em.merge(detached);check(m!=detached,"C04
merge returns managed copy");return null;});
  check(tx(em->em.find(CustomerEntity.class,id).getName()).equals("Merged"),"C04 merged
persisted");
 }
```

Expected check

Run the Maven project locally and confirm **C04 PASS**, then change the input to test a failure case.

Explain it in an interview

Verify the input reference did not become managed.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C05

Flush versus commit

Flush an UPDATE then roll back and prove it did not persist.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab05(){
  long id=customer();EntityManager em=emf.createEntityManager();

try{em.getTransaction().begin();em.find(CustomerEntity.class,id).setName("RolledBack");em.flush();
em.getTransaction().rollback();}
  finally{em.close();}
  check(!tx(x->x.find(CustomerEntity.class,id).getName()).equals("RolledBack"),"C05 flush not
commit");
 }
```

Expected check

Run the Maven project locally and confirm **C05 PASS**, then change the input to test a failure case.

Explain it in an interview

Separate SQL sent from durable transaction outcome.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C06

Cascade and orphan removal

Persist order lines through an aggregate root, then remove a line.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab06(){
  long c=customer();long o=order(c,"NEW",2);
  tx(em->{PurchaseOrder p=em.find(PurchaseOrder.class,o);check(p.getLines().size()==2,"C06 cascade
persist");
    p.removeFirstLine();return null;});
  check(tx(em->em.find(PurchaseOrder.class,o).getLines().size())==1,"C06 orphan removal");
 }
```

Expected check

Run the Maven project locally and confirm **C06 PASS**, then change the input to test a failure case.

Explain it in an interview

Inspect FK ownership and orphan deletion.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C07

JPQL projection

Bind a parameter and return a scalar view of an order.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab07(){
  long c=customer();long o=order(c,"DRAFT",0);
  String text=tx(em->em.createQuery("select o.status from PurchaseOrder o where
o.id=:id",String.class)
    .setParameter("id",o).getSingleResult());
  check(text.equals("DRAFT"),"C07 JPQL scalar projection");
 }
```

Expected check

Run the Maven project locally and confirm **C07 PASS**, then change the input to test a failure case.

Explain it in an interview

Contrast DTO/scalar query with managed entity tracking.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C08

Bounded join fetching

Use a fetch join for a known single-order detail screen.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab08(){
  long c=customer();long o=order(c,"FETCH",1);
  tx(em->{List<PurchaseOrder> rows=em.createQuery(
    "select distinct o from PurchaseOrder o join fetch o.customer left join fetch o.lines where
o.id=:id",PurchaseOrder.class)
    .setParameter("id",o).getResultList();
    check(rows.size()==1 && rows.get(0).getLines().size()==1,"C08 bounded join fetch");return
null;});
 }
```

Expected check

Run the Maven project locally and confirm **C08 PASS**, then change the input to test a failure case.

Explain it in an interview

Discuss row multiplication and pagination cautions.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C09

Optimistic locking conflict

Read one entity in two contexts and reject the stale second commit.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab09(){
  long id=customer();EntityManager a=emf.createEntityManager();EntityManager
b=emf.createEntityManager();
  try {a.getTransaction().begin();b.getTransaction().begin();
    CustomerEntity ca=a.find(CustomerEntity.class,id), cb=b.find(CustomerEntity.class,id);
    ca.setName("Winner");a.getTransaction().commit();cb.setName("Loser");
    boolean conflict=false;
    try {b.getTransaction().commit();}
    catch (RollbackException|OptimisticLockException expected){conflict=true;}
    check(conflict,"C09 optimistic lock must detect stale writer");
  }
finally{if(a.getTransaction().isActive())a.getTransaction().rollback();if(b.getTransaction().isAct
ive())b.getTransaction().rollback();a.close();b.close();}
 }
```

Expected check

Run the Maven project locally and confirm **C09 PASS**, then change the input to test a failure case.

Explain it in an interview

Retry the whole business decision, not merely a stale SQL statement.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C10

Bulk DML and stale state

Execute JPQL bulk UPDATE and refresh managed state.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab10(){
  long c=customer();long o=order(c,"BEFORE",0);
  tx(em->{PurchaseOrder loaded=em.find(PurchaseOrder.class,o);
    int n=em.createQuery("update PurchaseOrder o set o.status='AFTER' where o.id=:id")
      .setParameter("id",o).executeUpdate();check(n==1,"C10 bulk count");
    check(loaded.getStatus().equals("BEFORE"),"C10 context remains stale until refreshed");
    em.refresh(loaded);check(loaded.getStatus().equals("AFTER"),"C10 refresh");return null;});
 }
```

Expected check

Run the Maven project locally and confirm **C10 PASS**, then change the input to test a failure case.

Explain it in an interview

Bulk actions bypass entity dirty checking and callbacks.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C11

Conditional inventory update

Allow only one decrement of a single available unit.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab11(){
  long id=10000+next();tx(em->{em.persist(new InventoryEntity(id,1));return null;});
  tx(em->{int first=em.createQuery("update InventoryEntity i set i.available=i.available-1 where
i.id=:id and i.available>=1")
   .setParameter("id",id).executeUpdate();
   int second=em.createQuery("update InventoryEntity i set i.available=i.available-1 where
i.id=:id and i.available>=1")
   .setParameter("id",id).executeUpdate();check(first==1 && second==0,"C11 conditional stock
update");return null;});
 }
```

Expected check

Run the Maven project locally and confirm **C11 PASS**, then change the input to test a failure case.

Explain it in an interview

The database predicate enforces the invariant across JVM instances.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

HANDS-ON LAB C12

Detach stops tracking

Detach a managed entity and modify it without persistence.

COMPLETE DEMONSTRATION METHOD FROM LABRUNNER.JAVA

```
private static void lab12(){
  long id=customer();tx(em->{CustomerEntity c=em.find(CustomerEntity.class,id);
    check(em.contains(c),"C12 managed");em.detach(c);check(!em.contains(c),"C12 detached");
    c.setName("DetachedOnly");return null;});
  check(!tx(em->em.find(CustomerEntity.class,id).getName()).equals("DetachedOnly"),"C12 detached
change ignored");
 }
```

Expected check

Run the Maven project locally and confirm **C12 PASS**, then change the input to test a failure case.

Explain it in an interview

Reattach only intentionally; beware stale copies.

Execution status: Source supplied; Hibernate integration behavior requires a local Maven/Java run and is not claimed as verified in this environment.

30-DAY PLAN / PART 1

Daily interview preparation

45 minutes learning, 45 minutes hands-on coding, 20 minutes explaining and 10 minutes reviewing your mistakes.

DAY	FOCUS AND REQUIRED OUTPUT
01	ORM versus JPA; entity identity and lifecycle
02	Entity classes, IDs, generated values and embeddables
03	Managed/detached states; persist, merge, remove
04	Dirty checking, flush and commit
05	Many-to-one, ownership and mappedBy
06	Cascade, orphan removal and aggregate rules
07	Review 1: explain five mappings from memory
08	LAZY/EAGER, proxies and DTO boundaries
09	N+1 reproduction; choose fetch join
10	BatchSize, subselect and page-safe fetching

THE DAILY PRACTICE LOOP



30-DAY PLAN / PART 2

Daily interview preparation

45 minutes learning, 45 minutes hands-on coding, 20 minutes explaining and 10 minutes reviewing your mistakes.

DAY	FOCUS AND REQUIRED OUTPUT
11	JPQL parameters, named and typed queries
12	DTO projection and dynamic Criteria filters
13	Spring Specification and safe tenant scoping
14	Review 2: SQL counting and query-plan inspection
15	Spring @Transactional, rollback rules and self-invocation
16	Optimistic versions; simulate two writers
17	Pessimistic locks, deadlocks and timeouts
18	Cache levels, entity regions and invalidation
19	JDBC batching, IDENTITY and SEQUENCE tradeoffs
20	Bulk updates and persistence-context refresh

THE DAILY PRACTICE LOOP



30-DAY PLAN / PART 3

Daily interview preparation

45 minutes learning, 45 minutes hands-on coding, 20 minutes explaining and 10 minutes reviewing your mistakes.

DAY	FOCUS AND REQUIRED OUTPUT
21	Review 3: run labs C01-C06 with negatives
22	Inheritance strategies and mapping costs
23	Integration tests, H2 limitations and production DB
24	Migration discipline: Flyway/Liquibase + schema validate
25	Performance: pool waits, plans, row multiplication
26	Multi-tenant safety and bulk/native escape hatches
27	Order / payment workflow with outbox design
28	Run labs C07-C12, record evidence
29	Mock interview: explain one failure per pattern
30	Final mock: design, correctness, tuning and review

THE DAILY PRACTICE LOOP



LAST-MINUTE REFERENCE

10 Hibernate interview traps

01 flush != commit; a flushed change can roll back.

02 merge returns a managed instance; the input stays detached.

03 first-level cache is per persistence context, not globally shared.

04 LAZY does not prevent accidental queries during serialization.

05 JOIN FETCH with collections can make pagination unsafe.

06 N+1 should be measured with representative entity counts.

07 @Transactional self-invocation may skip proxy advice.

08 @Version does not protect every cross-row invariant.

09 Bulk DML can leave managed entities stale and bypass callbacks.

10 Cache annotations do not create durable or linearizable storage.

OFFICIAL READING

Study from primary sources

These references support the teaching baseline and should be consulted for exact API contracts and compatibility.

Hibernate ORM 6.6 User Guide
https://docs.hibernate.org/orm/6.6/userguide/html_single/

Hibernate ORM 6.6 documentation and compatibility
https://hibernate.org/orm/documentation/6.6/

Hibernate ORM 6.6 Migration Guide
https://docs.hibernate.org/orm/6.6/migration-guide/

Hibernate ORM 6.6 Getting Started
https://docs.hibernate.org/orm/6.6/quickstart/html_single/

Jakarta Persistence 3.1 API
https://jakarta.ee/specifications/persistence/3.1/

Spring Data JPA reference
https://docs.spring.io/spring-data/jpa/reference/

Publishing note

The explanations and questions in this edition were independently authored. The code package targets Hibernate ORM 6.6.58.Final, Jakarta Persistence 3.1 and H2. Test all examples with your intended dependency versions and database before commercial release.